

ROLE & EXPECTATION

You are a Senior AI Architect + MLOps Engineer + Data Scientist + Software Engineer combined.

Your task is to design a production-ready, end-to-end AI/ML + MLOps flagship project that will help me stand out for AI Engineer, ML Engineer, MLOps Engineer, and Data Scientist roles in top product, startup, and MNC companies.

You must think like a real industry expert and give a complete blueprint with no missing pieces.

PROJECT GOAL

Create a real-world, production-level AI project using advanced ML techniques such as:

RAG (Retrieval Augmented Generation)

AI Agents / Tool-Using Agents

LLMs (local or open source)

Classical ML / Deep Learning where useful

Automation pipelines

Evaluation + Monitoring

The project must:

Solve a real-world business problem

Be end-to-end

Be resume-worthy

Be deployable

Use mostly free / open-source tools

Avoid paid services unless free tiers are available

Be realistic for a single developer

CONSTRAINTS

Prefer free tools / open source

If cloud is used, it must be free tier compatible

Data must be public or freely available

No paid APIs

Must run on local laptop + optional cloud

Do not assume GPU availability

WHAT YOU MUST DELIVER

Give a step-by-step, phase-wise execution plan including:

1. Project Idea

Unique project title

Real-world use case

Why companies care

Resume-ready description

2. System Architecture

Text diagram

Component explanation

Data flow

Offline + Online pipeline

3. Tech Stack (Free / Open Source)

AI / ML

MLOps

Backend

Frontend

Deployment

Monitoring

Storage

CI/CD

4. Data Strategy

Exact data sources

How to collect

Storage method

Data versioning

Preprocessing pipeline

5. ML / AI Core

Embeddings

RAG pipeline

Agent design

Model choices

Prompt engineering

Evaluation metrics

Optimization techniques

6. MLOps Pipeline

Git workflow

DVC usage

MLflow experiment tracking

CI/CD pipeline

Docker setup

Testing strategy

Model registry

7. API Layer

FastAPI endpoints

Request/response schema

Auth (optional)

8. Frontend (Optional but Strong)

Streamlit / React

Features

UI ideas

9. Deployment

Free hosting options

Local Docker deployment

Cloud free tiers

Environment variables

10. Monitoring & Observability

Logging

Latency tracking

Error tracking

Model drift detection

Feedback loop

11. Advanced Add-Ons

Include optional but impressive features such as:

Multi-LLM routing

Auto retraining

Role-based access

Multi-language support

Vector DB switching

Caching

Async pipelines

Batch + Real-time inference

12. GitHub Repository Structure

Provide full folder structure and file names.

13. Timeline & Milestones

Weekly breakdown with realistic time estimates.

14. Resume Bullet Points

Give strong bullet points tailored for:

AI Engineer

ML Engineer

MLOps Engineer

Data Scientist

15. Interview Preparation

Possible interview questions from this project

Architecture questions

Trade-off discussions

OUTPUT RULES

Do not skip steps.

Be extremely detailed.

Assume beginner-to-intermediate but aiming for expert level.

Provide commands, tools, and example file names.

If the response is too long and token limit is reached, stop and I will type "continue" and you must continue from where you stopped without repeating content.

The project must be fully buildable with zero ambiguity.

Do not give generic answers; give implementation-level detail.

FINAL OBJECTIVE

By completing this project, I should be able to convincingly demonstrate skills in:

Machine Learning

Deep Learning

NLP / LLMs

RAG

AI Agents

Backend Engineering

MLOps

Deployment

Monitoring

System Design

This project should be strong enough to replace 3–4 small projects and be suitable for top-tier product companies. if at anywhere you are unable to complete total words just stop and ill write continue and start from there also do your best to find me a project

Thought for 17m 18s >

1. Project Idea

Unique project title

OpsPilot — Production-Grade Incident Response Copilot (AIOps + RAG + Tool-Using Agents)

Real-world use case

On-call/SRE/DevOps teams drown in alerts + logs and waste time:

- figuring out “what changed?”
- searching past incidents
- finding the right runbook
- writing incident updates + postmortems

OpsPilot is a deployable platform that:

1. **detects anomalies** in incoming logs (classical ML)
2. **retrieves** relevant runbooks + similar historical incidents (RAG)
3. uses a **tool-using agent** to orchestrate investigation steps and generate:
 - triage summary
 - likely root cause hypotheses
 - safe mitigation checklist
 - postmortem draft
4. captures **human feedback** and supports **auto-retraining** (optional)

Why companies care

- Outages cost money and reputation; reducing **MTTR** is a direct business KPI.
- Runbooks exist but are hard to search and often not followed consistently.
- AIOps (log anomaly detection + alert correlation) is a real industry domain.
- This project demonstrates **practical GenAI** integrated with **classical ML + MLOps + software engineering**.

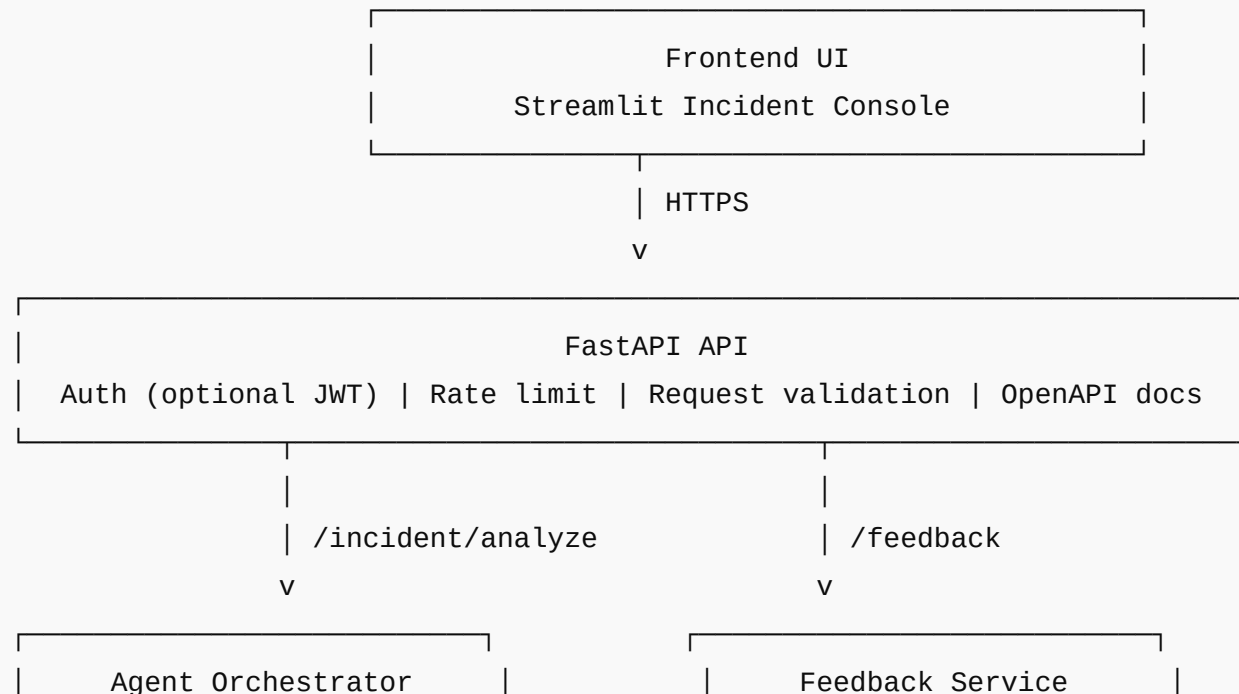
Resume-ready description (copy/paste)

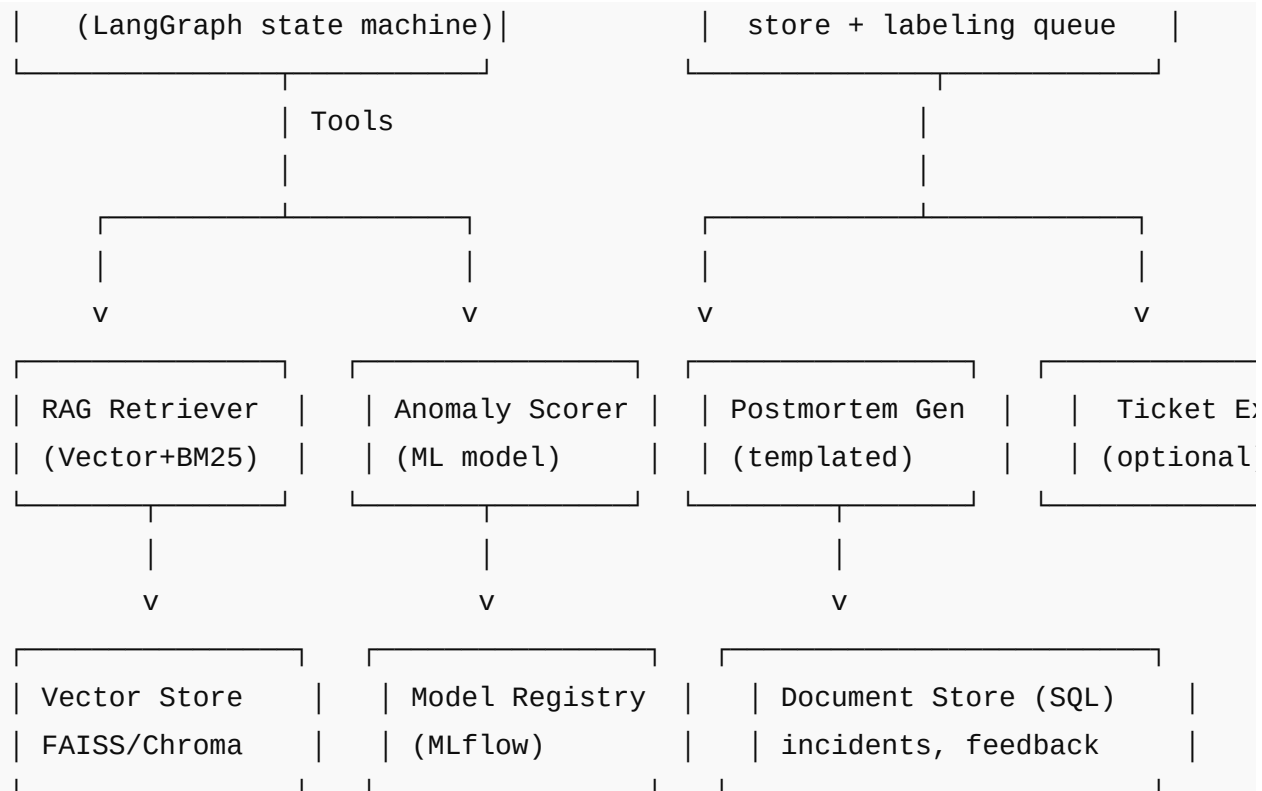
Built **OpsPilot**, an end-to-end **incident response copilot** combining **log anomaly detection (scikit-learn)**, **RAG over runbooks (FAISS/Chroma + SentenceTransformers)**, and a **tool-using LLM agent (LangGraph)** to generate triage summaries, mitigation plans, and postmortem drafts. Implemented full **MLOps** with **DVC**, **MLflow**, CI/CD via **GitHub Actions**, containerized deployment with **Docker Compose**, and production observability using **Prometheus/Grafana + structured logging + drift monitoring**.

2. System Architecture

Text diagram (production-style)

markdown





Observability:

- Prometheus metrics (API latency, token time, retrieval hit-rate)
- Grafana dashboards
- Structured logs (JSON)+ optional Loki
- Drift checks (Evidently) scheduled via Prefect

 Copy code

Component explanation (what each piece does)

- **FastAPI API:** single stable interface for UI + automation.
- **Agent Orchestrator (LangGraph):** deterministic workflow of steps:

- parse incident → score anomalies → retrieve evidence → decide next tool calls → generate response.
- **RAG Retriever:** indexes runbooks + incident docs; returns *grounded context*.
- **Anomaly Scorer:** classical ML detects abnormal log patterns; feeds “evidence” to the agent.
- **Stores:**
 - **Vector DB:** local FAISS/Chroma.
 - **SQL DB:** incidents, feedback, evaluation runs.
 - **MLflow:** experiment tracking + model registry.
- **Prefect:** scheduled ingestion, index rebuild, drift detection, optional retraining.

Data flow

Online (real-time inference)

1. User submits incident: alert text + time range + optional log snippet/file.
2. API normalizes + enriches request (service name, component tags).
3. Anomaly model scores log windows/templates.
4. Retriever pulls:
 - top-K similar incidents
 - top-K runbook sections
5. Agent composes a structured output:
 - incident summary
 - likely cause hypotheses
 - mitigation checklist
 - verification steps
 - postmortem draft (template-based + LLM polish)
6. User provides feedback → stored for evaluation + training.

Offline (batch pipelines)

- Download public data (logs + runbooks) → preprocess → train anomaly model → build embeddings → create vector index → evaluate → register best models → deploy.
-

3. Tech Stack (Free / Open Source)

AI / ML

- **Embeddings:** `sentence-transformers` (CPU-friendly)
 - model: `all-MiniLM-L6-v2` (fast on CPU)
- **Vector store:** **FAISS** (fast, local) or **Chroma** (simple persistence)
- **RAG framework:** **LlamaIndex** or **LangChain** (either is fine; I'll blueprint with LlamaIndex + LangGraph)
- **LLM (local, no paid APIs):**
 - **Ollama** (simplest local model runner) OR `llama.cpp` via `llama-cpp-python`
 - Use small instruction models (3B–7B) in quantized form for CPU
- **Classical ML / AIOps:**
 - `Drain3` for log template mining (streaming parser) [GitHub](#)
 - `scikit-learn` IsolationForest / OneClassSVM / LogisticRegression
 - Optional: `lightgbm` (CPU, fast)

MLOps

- **Data versioning:** DVC
- **Experiment tracking + registry:** MLflow
- **Orchestration:** Prefect (self-host/local)
- **Testing:** pytest

- **Lint/format:** ruff + black + mypy + pre-commit
- **CI/CD:** GitHub Actions

Backend

- FastAPI + Uvicorn
- Pydantic v2
- SQLAlchemy (or SQLModel) + Postgres/SQLite
- Redis (optional) for caching + background jobs

Frontend

- Streamlit (fastest to ship, very demo-friendly)

Deployment

- Docker + Docker Compose
- Optional free-tier hosting:
 - **Hugging Face Spaces (Docker)** supports deploying FastAPI containers

Hugging Face +1

- **Render** provides free instance types with limits Render

(You'll still keep full local Docker deployment as the default.)

Monitoring / Observability

- Prometheus + Grafana
- `prometheus-fastapi-instrumentator`
- Structured JSON logging (`structlog`)
- OpenTelemetry tracing (optional)
- Drift: Evidently (open-source)

4. Data Strategy

Exact data sources (public/free)

A) System logs for anomaly detection

- **Loghub (LogPAI):** collection of real-world system log datasets GitHub +1
 - Use **HDFS, BGL, OpenStack** subsets (good for anomaly detection tasks)

DeepWiki +1

- Repo: `logpai/loghub` GitHub

Note: Loghub is freely available for research/academic use and asks you to cite the repo/paper where applicable. Zenodo

B) Runbooks for RAG knowledge base

- **Prometheus-Operator Runbooks** (official runbooks repo, Apache-2.0) GitHub +1

This is perfect “real-world enterprise-style” documentation: Diagnosis, Mitigation, links, etc.

C) Optional “incident narratives” dataset (for richer postmortem generation)

- **PAG Synthetic Cloud Incident Dataset** (950 synthetic incidents + CAPMs) GitHub

Use it to:

- train/evaluate “action recommendation” quality
- test multi-step agent reasoning on realistic incident text

How to collect (implementation)

Create scripts:

- `scripts/data/download_loghub.py`
- `scripts/data/download_runbooks.py`

Example (Git-based download so it's reproducible):

```
bash

# logs
git clone https://github.com/logpai/loghub.git external/loghub

# runbooks
git clone https://github.com/prometheus-operator/runbooks.git external/runbooks
```

 Copy code

Then your scripts copy only required files into `data/raw/...` so DVC tracks a minimal subset.

Storage method

- `data/raw/` (DVC tracked)
- `data/processed/` (DVC tracked)
- `data/features/` (DVC tracked)
- SQL database for metadata:
 - `incidents`, `feedback`, `eval_runs`, `documents`

Data versioning (DVC)

```
bash

dvc init
dvc add data/raw data/processed data/features
git add data/.gitignore data/raw.dvc data/processed.dvc data/features.dvc
git commit -m "Track datasets with DVC"
```

 Copy code

DVC remote options (free):

- local folder remote (simplest)
- Google Drive remote (free)

(You can implement later; local remote is enough to demonstrate skill.)

Preprocessing pipeline (deterministic + reproducible)

1. **Log parsing:** Drain3 template mining → event IDs [GitHub](#)
2. **Sessionization:**
 - by timestamp windows (e.g., 5 min)
 - or by request/session ID if dataset provides it
3. **Feature extraction** per window:
 - template frequency vector
 - error-rate features
 - rare-template counts
4. **Label alignment** (if dataset has anomaly labels)
5. **Runbook ingestion:**
 - parse markdown
 - chunk by headings
 - store `doc_id`, `source`, `component`, `text`, `tags`
6. **Vector index build:**
 - embeddings → FAISS/Chroma persistence

5. ML / AI Core (RAG + Agents + Classical ML)

5.1 Embeddings

Model: sentence-transformers/all-MiniLM-L6-v2

Why:

- strong baseline
- CPU-friendly
- fast enough for laptop inference

Implementation file:

- `src/opspilot/embeddings/encoder.py`

Key design:

- batch embedding with caching (diskcache)
- store embedding version in metadata for future re-indexing

5.2 RAG pipeline (production-grade pattern)

Index schema

Each chunk stored as:

json

```
{
  "doc_id": "runbook:prometheus-operator:NodeFilesystemSpaceFillingUp#miti
  "source": "prometheus-operator-runbooks",
  "component": "node",
  "title": "NodeFilesystemSpaceFillingUp",
  "section": "Mitigation",
  "text": "...",
  "tags": ["kubernetes", "filesystem", "node"],
  "created_at": "...",
  "embedding_model": "all-MiniLM-L6-v2",
```

```
"checksum": "sha256..."  
}
```

 Copy code

Retrieval strategy (impressive but still buildable)

- **Hybrid retrieval:**
 - BM25 lexical retriever (fast, good for exact alert names)
 - Vector similarity (semantic)
- **Reranking (optional, CPU):**
 - cross-encoder reranker is heavy; instead do:
 - score fusion: $0.6 * \text{vector} + 0.4 * \text{bm25}$
 - or cheap heuristic rerank by matching alert name tokens

Context assembly

- hard cap context tokens by:
 - selecting top- $k=6$ chunks
 - and applying **context compression** (summarize long chunks with local small model if needed)

5.3 Classical ML: anomaly detection (AIOps)

Goal: produce *signal* and *evidence* for the agent.

Baseline approach (works without GPU):

- Drain3 → template IDs
- For each time window:
 - build vector of template frequencies (sparse → convert to dense with limited vocab)

- Train **IsolationForest** (unsupervised) or **LogisticRegression** (supervised if labels exist)

Files:

- `src/opspilot/anomaly/train.py`
- `src/opspilot/anomaly/infer.py`

Metrics:

- If labels exist: Precision/Recall/F1, PR-AUC
- If no labels: anomaly score stability + manual validation set

5.4 Agent design (LangGraph tool-using agent)

Agent roles

- **Incident Commander Agent** (main orchestrator)
- Optional sub-agents:
 - **Log Analyst Agent** (interprets anomaly outputs)
 - **Runbook Navigator Agent** (maps alert → runbook sections)
 - **Postmortem Writer Agent** (drafts final report)

Tooling (functions the agent can call)

- `tool_score_anomaly(log_payload, window_size) -> anomaly_report`
- `tool_retrieve_runbooks(query, filters) -> chunks`
- `tool_retrieve_similar_incidents(query) -> incidents`
- `tool_generate_postmortem(structured_fields) -> markdown`
- `tool_store_feedback(...)`

LangGraph state machine (deterministic + interview-friendly)

States:

1. `parse_request`
2. `anomaly_scan`
3. `retrieve_context`
4. `hypothesis_and_plan`
5. `draft_response`
6. `quality_check` (self-check: "is every recommendation grounded?")
7. `return_structured_output`

5.5 Model choices (LLM)

No paid APIs, CPU-only:

- Use **Ollama** (recommended for simplicity)
- Choose one small instruct model (example):
 - 3B instruct for speed
 - optional second bigger model for final response (multi-LLM routing add-on)

Deployment note: you can run the LLM locally in dev; for cloud demo you can ship a "small model mode" or disable generation and show retrieval/anomaly + templated responses if hosting constraints exist.

5.6 Prompt engineering (structured, testable)

Use *strict JSON schema output* for reliability.

System prompt goals:

- grounded in retrieved chunks
- never hallucinate commands
- produce structured sections

- cite evidence chunk IDs

Output schema example:

json

```
{
  "incident_summary": "...",
  "probable_causes": [{"cause": "...", "confidence": 0.0, "evidence": ["do"]},
  "recommended_actions": [{"step": "...", "risk": "low|med|high", "evidence": []}],
  "verification_steps": ["..."],
  "fallback_plan": ["..."],
  "postmortem_draft_markdown": "..."
}
```

 Copy code

5.7 Evaluation metrics (must-have)

Retrieval evaluation (RAG)

Create `data/eval/rag_gold.jsonl` mapping alert/incidents → expected runbook IDs.

Metrics:

- Recall@K
- MRR
- nDCG (optional)

Generation evaluation (offline)

Because no paid LLM judges:

- **Groundedness proxy:**
 - % of recommendations that reference at least one retrieved chunk ID

- **Faithfulness heuristic:**
 - overlap of key entities (alert name, component) between output and context
- **Human scoring UI** in Streamlit:
 - thumbs up/down + “wrong / unsafe / irrelevant” categories

Anomaly detection evaluation

- F1 / PR-AUC if labels exist
- alert-level false positive rate (FPR) on normal windows

5.8 Optimization techniques (CPU-friendly)

- Quantized LLM (4-bit) via Ollama/llama.cpp
 - Cache:
 - embeddings cache
 - retrieval results cache (Redis or diskcache)
 - Async tool calls (FastAPI background tasks for non-blocking ingestion)
 - Streaming responses from LLM (improves UX)
-

6. MLOps Pipeline (Git + DVC + MLflow + CI/CD + Docker)

6.1 Git workflow (simple but “real”)

Branches:

- `main` : always deployable
- `dev` : integration branch
- `feature/*` : short-lived
- `release/*` : optional

Rules:

- PR required to merge to `dev / main`
- CI must pass
- semantic version tags: `v0.1.0` , `v0.2.0`

6.2 DVC pipeline (reproducible DAG)

Key files:

- `dvc.yaml`
- `params.yaml`

Suggested stages:

1. `download_data`
2. `parse_logs`
3. `build_features`
4. `train_anomaly_model`
5. `build_rag_index`
6. `evaluate`
7. `package_artifacts`

Example `params.yaml` (minimal):

yaml

```
log_window_minutes: 5
anomaly:
  model: isolation_forest
  contamination: 0.02
rag:
```

```
top_k: 6
chunk_size: 800
chunk_overlap: 120
```

 Copy code

Example `dvc.yaml` skeleton:

yaml

```
stages:
  download_data:
    cmd: python scripts/data/download_all.py
    outs:
      - data/raw

  parse_logs:
    cmd: python scripts/features/parse_logs.py
    deps:
      - data/raw
    outs:
      - data/processed/log_templates.parquet

  build_features:
    cmd: python scripts/features/build_features.py
    deps:
      - data/processed/log_templates.parquet
    outs:
      - data/features/log_windows.parquet

  train_anomaly_model:
    cmd: python scripts/train/train_anomaly.py
    deps:
```

```
- data/features/log_windows.parquet
params:
- anomaly
outs:
- models/anomaly_model.pkl

build_rag_index:
cmd: python scripts/rag/build_index.py
deps:
- data/raw/runbooks
params:
- rag
outs:
- artifacts/vector_index

evaluate:
cmd: python scripts/eval/run_eval.py
deps:
- models/anomaly_model.pkl
- artifacts/vector_index
outs:
- reports/eval.json
```

 Copy code

Run everything:

```
bash
```

```
dvc repro
```

 Copy code

6.3 MLflow tracking + model registry

Start local MLflow:

```
bash
```

```
mlflow ui --backend-store-uri sqlite:///mlruns.db --default-artifact-root .
```

[Copy code](#)

Track:

- anomaly model params + metrics
- retrieval metrics
- packaged artifacts

Register model:

- `ospilot-anomaly-model`
- `ospilot-rag-index` (treat index as artifact version)

6.4 CI/CD (GitHub Actions)

Workflows:

- `ci.yml` : lint + tests + typecheck
- `build.yml` : docker build + security scan (optional)
- `release.yml` : tag -> build -> publish image (GitHub Container Registry)

`ci.yml` jobs:

- `ruff`
- `pytest`
- `mypy`
- `unit tests for retriever + agent graph` (mock LLM)

6.5 Docker setup (production-like)

Images:

- `ospilot-api`
- `ospilot-ui`

`docker-compose.yml` includes:

- `api`
- `ui`
- `postgres`
- `mlflow`
- `prometheus`
- `grafana`
- `redis` (optional)

6.6 Testing strategy (non-negotiable)

Tests to include:

- **Unit**
 - log parsing outputs expected schema
 - retriever returns top chunk for known query
 - anomaly scorer returns numeric score
- **Contract**
 - FastAPI OpenAPI schema snapshot
- **Integration**
 - spin up docker compose, call `/incident/analyze`
- **Safety**
 - agent output must contain `evidence` for each action

7. API Layer (FastAPI)

Endpoints (must implement)

Core

- GET /health
- POST /incident/analyze
- POST /rag/search
- POST /anomaly/score
- POST /feedback

Ops / Admin

- POST /admin/reindex (protected)
- POST /admin/retrain (optional, protected)
- GET /metrics (Prometheus)

Example request/response schemas

POST /incident/analyze

Request (IncidentRequest):

json

```
{
  "incident_id": "INC-2026-00012",
  "alert_title": "NodeFilesystemSpaceFillingUp",
  "service": "kubernetes",
  "environment": "prod",
  "log_lines": ["...", "..."],
```

```
"time_range": {"start": "2026-02-07T10:00:00Z", "end": "2026-02-07T10:10"},
"user_notes": "Pods failing scheduling"
}
```

 Copy code

Response (`IncidentAnalysisResponse`):

json

```
{
  "summary": "...",
  "anomaly_report": {"score": 0.91, "top_templates": ["..."]},
  "retrieved_context": [{"doc_id": "...", "title": "..."}],
  "actions": [{"step": "...", "risk": "low", "evidence": ["doc_id..."]}],
  "postmortem_markdown": "..."
}
```

 Copy code

Auth (optional but impressive)

- JWT auth with `Authorization: Bearer <token>`
- RBAC roles:
 - `viewer`
 - `responder`
 - `admin`

8. Frontend (Optional but Strong): Streamlit Incident Console

Features

- Incident intake form (alert + logs + notes)
- Live “analysis trace” panel:
 - anomaly score
 - retrieved runbook sections
 - agent decisions (LangGraph state transitions)
- Output tabs:
 - Triage Summary
 - Recommended Actions
 - Verification
 - Postmortem Draft
- Feedback capture:
 - 👍/👎
 - “unsafe suggestion”, “irrelevant”, “missing runbook”, “wrong root cause”

UI ideas that impress interviewers

- “Evidence view”: clicking an action shows the exact runbook chunk(s) backing it
- “Time window anomaly plot”: anomaly score across windows
- “Retrieval debugging”: show BM25 vs Vector scores

9. Deployment

Local Docker deployment (default)

```
bash
```

```
docker compose up --build  
# UI: http://localhost:8501
```

```
# API: http://localhost:8000/docs  
# Grafana: http://localhost:3000
```

 Copy code

Free hosting options (optional)

Option A: Hugging Face Spaces (Docker)

- Deploy FastAPI (and even Streamlit) using Spaces Docker support Hugging Face +1
Best for portfolio demos because it's Git-based and simple.

Option B: Render free instances

Render documentation states free instance types exist (with limits) Render

Good for hosting FastAPI.

Recommendation: for a portfolio, deploy UI + API together on one platform if possible, and keep the full “compose stack” runnable locally.

Environment variables (`.env.example`)

- `DATABASE_URL=postgresql+psycopg://...`
- `VECTOR_STORE=faiss`
- `EMBED_MODEL_NAME=all-MiniLM-L6-v2`
- `LLM_PROVIDER=ollama`
- `OLLAMA_HOST=http://ollama:11434`
- `ENABLE_AUTH=false`
- `JWT_SECRET=...`
- `MLFLOW_TRACKING_URI=http://mlflow:5000`

10. Monitoring & Observability

Logging

- Structured JSON logs with:
 - `request_id`
 - `incident_id`
 - `latency_ms`
 - `retrieval_k`
 - `anomaly_score`
 - `llm_tokens_estimated` (approx)

Latency tracking (Prometheus)

Track histograms:

- API endpoint latency
- retrieval latency
- embedding latency
- LLM generation latency

Error tracking

- Store exception traces
- Count tool failures per tool
- Circuit breaker for LLM timeouts (fallback to templated response)

Model drift detection (Evidently + scheduled job)

- Drift on:
 - log template distribution
 - anomaly score distribution
 - embedding centroid drift

- Trigger:
 - “retrain recommended” notification
 - optional auto-retrain pipeline (advanced add-on)

Feedback loop

- Feedback stored in SQL with:
 - `helpful` boolean
 - tags
 - corrected root cause (optional)
 - Weekly job:
 - produce `reports/feedback_summary.md`
 - update evaluation dataset
-

11. Advanced Add-Ons (Optional but Very Impressive)

Pick 3–5 of these; don’t do all at once.

1. Multi-LLM routing

- small model for classification + planning
 - bigger model for final postmortem draft
- Routing rule: based on input length + severity.

2. Vector DB switching

- abstraction layer: FAISS (default), Chroma, optional Qdrant (docker)
- show benchmark script: `scripts/bench/vectorstore_bench.py`

3. Caching

- Redis cache for:
 - embeddings

- retrieval results
- repeated incident analyses

4. Async pipelines

- ingestion/reindex done in background via Celery/Redis
- FastAPI returns `job_id`, UI polls status

5. Batch + real-time inference

- Batch: nightly processing of “yesterday logs”
- Real-time: analyze an uploaded snippet instantly

6. Role-based access

- Admin-only `/admin/retrain` and `/admin/reindex`

7. Multi-language support

- detect language, translate to English for retrieval, translate answer back (open-source translation)

12. GitHub Repository Structure (full structure + filenames)

pgsql

opspilot/

- ├─ README.md
- ├─ LICENSE
- ├─ .env.example
- ├─ .gitignore
- ├─ pyproject.toml
- ├─ uv.lock / poetry.lock (choose one)
- ├─ Makefile
- ├─ docker-compose.yml

```
├─ docker/
│   ├── api.Dockerfile
│   ├── ui.Dockerfile
│   └─ prometheus.yml
├─ .github/
│   └─ workflows/
│       ├── ci.yml
│       ├── build.yml
│       └─ release.yml
├─ configs/
│   ├── logging.yaml
│   ├── model_registry.yaml
│   └─ rbac.yaml
├─ data/                                # DVC-tracked
│   ├── raw/
│   │   ├── loghub_subset/
│   │   └─ runbooks/
│   ├── processed/
│   └─ features/
├─ artifacts/
│   └─ vector_index/
├─ models/
│   └─ anomaly_model.pkl
├─ reports/
│   ├── eval.json
│   ├── drift_report.html
│   └─ feedback_summary.md
├─ scripts/
│   ├── data/
│   │   └─ download_all.py
```

```
| | | └─ download_loghub.py
| | |   └─ download_runbooks.py
| | └─ features/
| | | └─ parse_logs.py
| | |   └─ build_features.py
| | └─ train/
| | |   └─ train_anomaly.py
| | └─ rag/
| | |   └─ build_index.py
| | └─ eval/
| | |   └─ run_eval.py
└─ src/
  └─ opspilot/
    └─ __init__.py
    └─ api/
      └─ main.py
      └─ routes/
        └─ health.py
        └─ incident.py
        └─ rag.py
        └─ anomaly.py
        └─ feedback.py
      └─ schemas.py
      └─ deps.py
    └─ agent/
      └─ graph.py
      └─ tools.py
      └─ prompts.py
      └─ safety.py
    └─ rag/
```

```
|      |  ├── index.py
|      |  ├── retriever.py
|      |  ├── chunking.py
|      |  └── bm25.py
|      ├── anomaly/
|      |  ├── train.py
|      |  ├── infer.py
|      |  └── features.py
|      ├── storage/
|      |  ├── db.py
|      |  └── models.py
|      ├── observability/
|      |  ├── metrics.py
|      |  └── logging.py
|      └── workflows/
|          ├── prefect_flows.py
|          └── drift.py
└── ui/
    └── streamlit_app.py
└── tests/
    ├── test_api_contract.py
    ├── test_retrieval.py
    ├── test_anomaly.py
    ├── test_agent_safety.py
    └── test_end_to_end.py
```

 Copy code

13. Timeline & Milestones (weekly breakdown)

Week 1 — Foundation (6–10 hrs)

- Repo scaffolding + tooling
- Docker Compose skeleton
- DVC init + first dataset download scripts

Deliverable: `docker compose up` brings up API/UI skeleton + empty pipeline

Week 2 — Data ingestion + preprocessing (8–12 hrs)

- Loghub subset download + parsing into parquet
- Runbooks ingestion + markdown chunking

Deliverable: `dvc repro parse_logs build_rag_index`

Week 3 — Anomaly model (8–14 hrs)

- Drain3 templates + window features
- Train IsolationForest baseline + eval report
- Log artifacts to MLflow

Deliverable: `models/anomaly_model.pkl` + `reports/eval.json` (anomaly)

Week 4 — RAG retrieval + evaluation (8–14 hrs)

- Hybrid retriever + top-k evaluation
- “Evidence view” metadata

Deliverable: retrieval metrics + debug UI tab

Week 5 — Agent orchestration (10–16 hrs)

- LangGraph workflow + tools
- Safety rules (“no destructive commands”)
- Structured JSON outputs

Deliverable: `/incident/analyze` working end-to-end

Week 6 — Production hardening (10–16 hrs)

- tests (unit + integration)
- Prometheus metrics + Grafana dashboard
- structured logging + request IDs

Deliverable: reliable demo with observability

Week 7 — Deployment + CI/CD (8–14 hrs)

- GitHub Actions CI
- container build + push (optional)
- optional HF Spaces/Render deploy

Deliverable: public demo link + reproducible local run

Week 8 — Advanced add-ons + polish (8–14 hrs)

- caching + async jobs OR drift detection + retrain trigger
- README, architecture docs, screenshots

Deliverable: “flagship-grade” GitHub project

14. Resume Bullet Points (role-tailored)

AI Engineer

- Built a **tool-using incident response agent** (LangGraph) that orchestrates anomaly scoring + RAG retrieval to generate **grounded mitigation plans** with evidence links.
- Implemented **hybrid RAG** (BM25 + vector) over real runbooks to improve Recall@K and reduce hallucinations via structured outputs + safety checks.

ML Engineer

- Developed a CPU-efficient **log anomaly detection pipeline** using **Drain3 template mining**  and scikit-learn, with reproducible preprocessing and evaluation.
- Designed offline evaluation harness for retrieval + anomaly detection, enabling regression testing across data/model versions.

MLOps Engineer

- Productionized training + indexing with **DVC pipelines**, tracked experiments in **MLflow**, and shipped containerized services using **Docker Compose** with CI via **GitHub Actions**.
- Added observability: Prometheus metrics, Grafana dashboards, structured logging, and scheduled drift checks feeding retraining recommendations.

Data Scientist

- Performed feature engineering on log template sequences and evaluated anomaly detection using PR-AUC/F1 and operational false-positive constraints.
- Built feedback-driven evaluation loops capturing human judgments and converting them into labeled datasets for future model improvements.

15. Interview Preparation (questions + trade-offs)

Architecture questions

1. Why use an agent + tools instead of “one big prompt”?
2. How do you guarantee groundedness in RAG outputs?
3. Where do you cache and why? What are cache invalidation rules?
4. How do you design for offline + online separation?

RAG / LLM questions

1. Chunking strategy: by heading vs fixed-size—trade-offs?
2. Why hybrid retrieval? When does BM25 beat embeddings?
3. How do you evaluate RAG without paid LLM judges?
4. How do you prevent prompt injection from runbook text?

ML / anomaly detection questions

1. IsolationForest vs OneClassSVM—when to use which?
2. What features matter for log anomaly detection?
3. How do you handle concept drift (new templates)?

MLOps questions

1. Why DVC vs Git LFS? How do you structure DVC stages?
2. What does “model registry” mean for non-model artifacts like a vector index?
3. What CI tests prevent silent quality regressions?

Trade-off discussions (talk like a senior)

- **CPU-only constraint** → choose small quantized LLM; rely more on retrieval + structured templates.
- **Vector DB choice** → FAISS for speed + simplicity; Chroma for persistence; Qdrant for production-like API.
- **Safety** → evidence-backed actions, allowlist of “safe commands”, and explicit uncertainty formatting.

If you type “**continue**”, I can add (without repeating anything):

- a concrete `docker-compose.yml` template (API + Postgres + MLflow + Prometheus + Grafana)
- a minimal FastAPI `main.py` + Pydantic schemas
- a LangGraph `graph.py` skeleton with tool definitions
- exact Makefile targets (`make data`, `make train`, `make index`, `make serve`, `make test`)

   ...   Sources

continue,

16. Concrete implementation templates (copy/paste-ready)

16.1 `docker-compose.yml` (API + UI + Postgres + MLflow + Prometheus + Grafana + Redis + Ollama optional)

yaml

```
version: "3.9"

services:
  postgres:
    image: postgres:16
    environment:
      POSTGRES_USER: opspilot
      POSTGRES_PASSWORD: opspilot
```

```
    POSTGRES_DB: opspilot
  ports:
    - "5432:5432"
  volumes:
    - pg_data:/var/lib/postgresql/data

mlflow:
  image: ghcr.io/mlflow/mlflow:v2.14.3
  command: >
    mlflow server
    --backend-store-uri postgresql://opspilot:opspilot@postgres:5432/opspilot
    --default-artifact-root /mlruns
    --host 0.0.0.0 --port 5000
  environment:
    MLFLOW_TRACKING_URI: http://localhost:5000
  depends_on:
    - postgres
  ports:
    - "5000:5000"
  volumes:
    - mlruns:/mlruns

redis:
  image: redis:7
  ports:
    - "6379:6379"

# Optional local LLM runtime (CPU-friendly with small quantized models)
# If you don't want it, comment this service out and set LLM_PROVIDER=moonshot
ollama:
```

```
image: ollama/ollama:latest
ports:
  - "11434:11434"
volumes:
  - ollama:/root/.ollama

api:
  build:
    context: .
    dockerfile: docker/api.Dockerfile
  env_file:
    - .env
  environment:
    DATABASE_URL: postgresql+psycpg://opspilot:opspilot@postgres:5432/op
    MLFLOW_TRACKING_URI: http://mlflow:5000
    REDIS_URL: redis://redis:6379/0
    OLLAMA_HOST: http://ollama:11434
  depends_on:
    - postgres
    - mlflow
    - redis
    - ollama
  ports:
    - "8000:8000"

ui:
  build:
    context: .
    dockerfile: docker/ui.Dockerfile
  env_file:
```

```
- .env
environment:
  API_BASE_URL: http://api:8000
depends_on:
  - api
ports:
  - "8501:8501"

prometheus:
  image: prom/prometheus:v2.53.1
  volumes:
    - ./docker/prometheus.yml:/etc/prometheus/prometheus.yml:ro
  ports:
    - "9090:9090"
  depends_on:
    - api

grafana:
  image: grafana/grafana:11.1.0
  ports:
    - "3000:3000"
  volumes:
    - grafana:/var/lib/grafana
  depends_on:
    - prometheus

volumes:
  pg_data:
  mlruns:
```

 Copy code

grafana:

ollama:

16.2 docker/api.Dockerfile

dockerfile

```
FROM python:3.11-slim
```

```
ENV PYTHONDONTWRITEBYTECODE=1 \
    PYTHONUNBUFFERED=1
```

```
WORKDIR /app
```

```
# System deps for FAISS / basic builds (keep minimal)
```

```
RUN apt-get update && apt-get install -y --no-install-recommends \
    build-essential curl git \
    && rm -rf /var/lib/apt/lists/*
```

```
# Install deps
```

```
COPY pyproject.toml /app/pyproject.toml
```

```
# If using uv:
```

```
# RUN pip install uv && uv pip install --system -r pyproject.toml
```

```
RUN pip install --upgrade pip && pip install -e ".[api]"
```

```
# Copy source
```

```
COPY src /app/src
```

```
COPY scripts /app/scripts
```

```
COPY artifacts /app/artifacts
```

```
COPY models /app/models
```

```
EXPOSE 8000
```

```
CMD ["uvicorn", "opspilot.api.main:app", "--host", "0.0.0.0", '📄 Copy code 30
```

16.3 docker/ui.Dockerfile

dockerfile

```
FROM python:3.11-slim
```

```
WORKDIR /app
```

```
RUN pip install --upgrade pip
```

```
COPY pyproject.toml /app/pyproject.toml
```

```
RUN pip install -e ".[ui]"
```

```
COPY ui /app/ui
```

```
EXPOSE 8501
```

```
CMD ["streamlit", "run", "ui/streamlit_app.py", "--server.port=8501", "--s
```

📄 Copy code

16.4 docker/prometheus.yml

yaml

```
global:
  scrape_interval: 10s

scrape_configs:
  - job_name: "opspilot-api"
    metrics_path: "/metrics"
    static_configs:
      - targets: ["api:8000"]
```

 Copy code

17. Python dependencies (pyproject.toml minimal, Poetry-style)

Use whichever tool you like (Poetry / uv / pip). Here's a workable pyproject.toml (Poetry format):

toml

```
[tool.poetry]
name = "opspilot"
version = "0.1.0"
description = "Incident Response Copilot (AIOps + RAG + Agents)"
authors = ["you <you@example.com>"]
packages = [{ include = "opspilot", from = "src" }]

[tool.poetry.dependencies]
python = "^3.11"
fastapi = "^0.112.0"
uvicorn = {extras = ["standard"], version = "^0.30.0"}
```

```
pydantic = "^2.8.2"
sqlmodel = "^0.0.21"
psycopg = {extras = ["binary"], version = "^3.2.1"}
httpx = "^0.27.0"
numpy = "^2.0.1"
pandas = "^2.2.2"
scikit-learn = "^1.5.1"
joblib = "^1.4.2"
pyyaml = "^6.0.2"
structlog = "^24.2.0"
prometheus-fastapi-instrumentator = "^7.0.0"
redis = "^5.0.7"
diskcache = "^5.6.3"

# RAG + embeddings
sentence-transformers = "^3.0.1"
faiss-cpu = "^1.8.0.post1"
rank-bm25 = "^0.2.2"

# Agents
langgraph = "^0.2.25"
langchain-core = "^0.2.34"

# Optional orchestration/monitoring
prefect = "^2.20.1"
evidently = "^0.4.36"
mlflow = "^2.14.3"

[tool.poetry.group.dev.dependencies]
pytest = "^8.3.2"
```

```
pytest-asyncio = "^0.23.8"
ruff = "^0.5.7"
mypy = "^1.11.1"
types-PyYAML = "^6.0.12.20240808"

[tool.poetry.extras]
api = []
ui = ["streamlit"]

[build-system]
requires = ["poetry-core"]
build-backend = "poetry.core.masonry.api"
```

 Copy code

18. `.env.example` (copy to `.env`)

```
bash

# Core
ENV=dev
LOG_LEVEL=INFO

# Storage
DATABASE_URL=sqlite:///./opspilot.db
REDIS_URL=redis://localhost:6379/0

# RAG
EMBED_MODEL_NAME=sentence-transformers/all-MiniLM-L6-v2
VECTOR_INDEX_PATH=./artifacts/vector_index
RAG_TOP_K=6
```

```
# LLM
LLM_PROVIDER=ollama # ollama|mock
OLLAMA_HOST=http://localhost:11434
OLLAMA_MODEL=llama3.2:3b-instruct-q4_K_M

# API/UI
API_BASE_URL=http://localhost:8000

# Auth (optional)
ENABLE_AUTH=false
JWT_SECRET=change_me

# MLflow
MLFLOW_TRACKING_URI=http://localhost:5000
```

 Copy code

19. FastAPI backend skeleton

19.1 `src/opspilot/api/main.py`

python

```
from fastapi import FastAPI
from opspilot.api.routes.health import router as health_router
from opspilot.api.routes.incident import router as incident_router
from opspilot.api.routes.rag import router as rag_router
from opspilot.api.routes.anomaly import router as anomaly_router
from opspilot.api.routes.feedback import router as feedback_router
from opspilot.observability.logging import configure_logging
```

```
from opspilot.observability.metrics import instrument_app

def create_app() -> FastAPI:
    configure_logging()
    app = FastAPI(title="OpsPilot API", version="0.1.0")

    app.include_router(health_router)
    app.include_router(incident_router, prefix="/incident", tags=["incident"])
    app.include_router(rag_router, prefix="/rag", tags=["rag"])
    app.include_router(anomaly_router, prefix="/anomaly", tags=["anomaly"])
    app.include_router(feedback_router, prefix="/feedback", tags=["feedback"])

    instrument_app(app)
    return app

app = create_app()
```

 Copy code

19.2 src/ospilot/api/schemas.py

python

```
from pydantic import BaseModel, Field
from typing import List, Optional, Literal, Dict, Any

class TimeRange(BaseModel):
    start: str
    end: str

class IncidentRequest(BaseModel):
    incident_id: str
```

```
    alert_title: str
    service: Optional[str] = None
    environment: Optional[str] = None
    log_lines: List[str] = Field(default_factory=list)
    time_range: Optional[TimeRange] = None
    user_notes: Optional[str] = None

class RagSearchRequest(BaseModel):
    query: str
    filters: Dict[str, Any] = Field(default_factory=dict)
    top_k: int = 6

class RetrievedChunk(BaseModel):
    doc_id: str
    title: str
    section: Optional[str] = None
    text: str
    score: float

class AnomalyScoreRequest(BaseModel):
    log_lines: List[str]
    window_minutes: int = 5

class AnomalyReport(BaseModel):
    score: float
    top_templates: List[str] = Field(default_factory=list)
    details: Dict[str, Any] = Field(default_factory=dict)

Risk = Literal["low", "med", "high"]
```



```
class RecommendedAction(BaseModel):
    step: str
    risk: Risk
    evidence: List[str] = Field(default_factory=list)

class IncidentAnalysisResponse(BaseModel):
    summary: str
    anomaly_report: AnomalyReport
    retrieved_context: List[RetrievedChunk]
    actions: List[RecommendedAction]
    verification_steps: List[str]
    fallback_plan: List[str]
    postmortem_markdown: str
    trace: Dict[str, Any] = Field(default_factory=dict)

class FeedbackRequest(BaseModel):
    incident_id: str
    helpful: bool
    tags: List[str] = Field(default_factory=list)
    comment: Optional[str] = None
    suggested_root_cause: Optional[str] = None
```

 Copy code

19.3 src/ospilot/api/routes/health.py

python

```
from fastapi import APIRouter

router = APIRouter()
```

```
@router.get("/health")
def health():
    return {"status": "ok"}
```

 Copy code

20. RAG implementation (FAISS + BM25 hybrid)

20.1 src/opspilot/rag/chunking.py

python

```
from dataclasses import dataclass
from typing import List

@dataclass
class Chunk:
    doc_id: str
    title: str
    section: str
    text: str
    metadata: dict

def simple_md_chunk(title: str, section: str, text: str, chunk_size: int =
    # naive chunker: safe, deterministic
    chunks = []
    i = 0
    while i < len(text):
        chunks.append(text[i:i+chunk_size])
```

```
i += max(1, chunk_size - overlap)
return chunks
```

 Copy code

20.2 src/opspilot/rag/bm25.py

python

```
from rank_bm25 import BM25Okapi
from typing import List, Tuple
import re

def tokenize(text: str) -> List[str]:
    return re.findall(r"[A-Za-z0-9_:/.-]+", text.lower())

class BM25Index:
    def __init__(self, texts: List[str], ids: List[str]):
        self.ids = ids
        self.tokens = [tokenize(t) for t in texts]
        self.bm25 = BM25Okapi(self.tokens)

    def search(self, query: str, top_k: int = 6) -> List[Tuple[str, float]]:
        scores = self.bm25.get_scores(tokenize(query))
        pairs = list(zip(self.ids, scores))
        pairs.sort(key=lambda x: x[1], reverse=True)
        return pairs[:top_k]
```

 Copy code

20.3 src/opspilot/embeddings/encoder.py

python

```
from sentence_transformers import SentenceTransformer
from diskcache import Cache
from typing import List
import numpy as np
import os
import hashlib

class Embedder:
    def __init__(self, model_name: str, cache_dir: str = ".cache/embedding:
        self.model = SentenceTransformer(model_name)
        os.makedirs(cache_dir, exist_ok=True)
        self.cache = Cache(cache_dir)
        self.model_name = model_name

    def _key(self, text: str) -> str:
        h = hashlib.sha256((self.model_name + "||" + text).encode("utf-8"))
        return h

    def encode(self, texts: List[str]) -> np.ndarray:
        vecs = []
        missing = []
        missing_idx = []

        for i, t in enumerate(texts):
            k = self._key(t)
            v = self.cache.get(k)
            if v is None:
                missing.append(t)
                missing_idx.append(i)
                vecs.append(None)
```

```

        else:
            vecs.append(v)

    if missing:
        enc = self.model.encode(missing, normalize_embeddings=True, batch_size=batch_size)
        for j, i in enumerate(missing_idx):
            self.cache.set(self._key(texts[i]), enc[j])
            vecs[i] = enc[j]

    return np.vstack(vecs).astype("float32")

```

 Copy code

20.4 src/opspilot/rag/index.py (FAISS + metadata store)

python

```

import os, json
import numpy as np
import faiss
from typing import List, Dict, Any, Tuple
from opspilot.embeddings.encoder import Embedder

class FaissStore:
    def __init__(self, index_path: str, embed_model_name: str):
        self.index_path = index_path
        self.embedder = Embedder(embed_model_name)
        self.faiss_file = os.path.join(index_path, "index.faiss")
        self.meta_file = os.path.join(index_path, "meta.jsonl")
        self._index = None
        self._meta: List[Dict[str, Any]] = []

```

```
def load(self):
    if os.path.exists(self.faiss_file):
        self._index = faiss.read_index(self.faiss_file)
    else:
        self._index = faiss.IndexFlatIP(384) # MiniLM dims=384
    if os.path.exists(self.meta_file):
        self._meta = []
        with open(self.meta_file, "r", encoding="utf-8") as f:
            for line in f:
                self._meta.append(json.loads(line))
    return self

def build(self, docs: List[Dict[str, Any]]):
    os.makedirs(self.index_path, exist_ok=True)
    texts = [d["text"] for d in docs]
    vecs = self.embedder.encode(texts)
    self._index = faiss.IndexFlatIP(vecs.shape[1])
    self._index.add(vecs)
    self._meta = docs

    faiss.write_index(self._index, self.faiss_file)
    with open(self.meta_file, "w", encoding="utf-8") as f:
        for d in docs:
            f.write(json.dumps(d, ensure_ascii=False) + "\n")

def search(self, query: str, top_k: int = 6) -> List[Tuple[Dict[str, Any], float]]:
    qv = self.embedder.encode([query])
    scores, idxs = self._index.search(qv, top_k)
    results = []
    for score, idx in zip(scores[0], idxs[0]):
```

```

        if idx == -1:
            continue
        results.append((self._meta[idx], float(score)))
    return results

```

 Copy code

20.5 src/opspilot/rag/retriever.py (hybrid fusion)

python

```

from typing import List, Dict, Any
from opspilot.rag.index import FaissStore
from opspilot.rag.bm25 import BM25Index

class HybridRetriever:
    def __init__(self, faiss_store: FaissStore, bm25: BM25Index, alpha: float):
        self.faiss = faiss_store
        self.bm25 = bm25
        self.alpha = alpha  # weight for vector score

    def search(self, query: str, top_k: int = 6) -> List[Dict[str, Any]]:
        vec_hits = self.faiss.search(query, top_k=top_k * 3)
        bm_hits = self.bm25.search(query, top_k=top_k * 3)

        # Normalize scores
        vec_scores = {h[0]["doc_id"]: h[1] for h in vec_hits}
        bm_scores = {doc_id: score for doc_id, score in bm_hits}

        all_ids = set(vec_scores) | set(bm_scores)
        if not all_ids:
            return []

```

```
def norm(d):
    if not d:
        return {}
    mx = max(d.values()) or 1.0
    return {k: v / mx for k, v in d.items()}

nvec = norm(vec_scores)
nbm = norm(bm_scores)

fused = []
meta_by_id = {h[0]["doc_id"]: h[0] for h in vec_hits}
# bm25-only docs won't have meta here in this simplified template;
# in production, keep a docstore by id for both indexes.
for doc_id in all_ids:
    score = self.alpha * nvec.get(doc_id, 0.0) + (1 - self.alpha)
    meta = meta_by_id.get(doc_id)
    if meta:
        fused.append((meta, score))

fused.sort(key=lambda x: x[1], reverse=True)
return [
    {
        "doc_id": m["doc_id"],
        "title": m.get("title", ""),
        "section": m.get("section", ""),
        "text": m["text"],
        "score": float(s),
    }
]
```



```
        for m, s in fused[:top_k]  
    ]
```

 Copy code

21. Anomaly detection pipeline (Drain3 + IsolationForest)

21.1 `scripts/features/parse_logs.py` (template mining)

python

```
import os  
import pandas as pd  
from drain3 import TemplateMiner  
from drain3.template_miner_config import TemplateMinerConfig  
  
INPUT = "data/raw/loghub_subset/hdfs/HDFS.log"  
OUT = "data/processed/log_templates.parquet"  
  
def main():  
    os.makedirs("data/processed", exist_ok=True)  
  
    cfg = TemplateMinerConfig()  
    cfg.load("drain3.ini") # you create this config file (see below)  
    miner = TemplateMiner(cfg)  
  
    rows = []  
    with open(INPUT, "r", encoding="utf-8", errors="ignore") as f:  
        for i, line in enumerate(f):  
            result = miner.add_log_message(line.strip())
```

```

        template = result["template_mined"]
        cluster_id = result["cluster_id"]
        rows.append({"line": line.strip(), "template": template, "cluster_id": cluster_id})
        if i > 200000: # keep subset manageable for laptop demo
            break

df = pd.DataFrame(rows)
df.to_parquet(OUT, index=False)
print(f"Wrote {OUT} rows={len(df)}")

if __name__ == "__main__":
    main()

```

 Copy code

21.2 drain3.ini (put at repo root)

```

ini

[SNAPSHOT]
snapshot_interval_minutes=10
compress_state=true

[MASKING]
masking=[{"regex_pattern": "\\b\\d+\\b", "mask_with": "NUM"},
          {"regex_pattern": "\\b[0-9a-f]{8}\\b", "mask_with": "HEX"}]

[DRAIN]
sim_th=0.4
depth=4
max_children=100

```

```
max_clusters=100000  
extra_delimiters=["_", "-", "/", "."]
```

 Copy code

21.3 scripts/features/build_features.py (windowed template counts)

python

```
import os  
import pandas as pd  
import numpy as np  
from collections import Counter  
  
INP = "data/processed/log_templates.parquet"  
OUT = "data/features/log_windows.parquet"  
  
def main():  
    os.makedirs("data/features", exist_ok=True)  
    df = pd.read_parquet(INP)  
  
    # For demo: fake "time windows" by chunking sequentially  
    window_size = 200  
    windows = []  
    vocab = df["cluster_id"].value_counts().head(300).index.tolist()  
    vocab_set = set(vocab)  
  
    for w, start in enumerate(range(0, len(df), window_size)):  
        chunk = df.iloc[start:start+window_size]  
        cnt = Counter([c for c in chunk["cluster_id"].tolist() if c in vocab])  
        vec = [cnt.get(c, 0) for c in vocab]  
        windows.append({"window_id": w, "vec": vec})
```

```

out = pd.DataFrame(windows)
out.to_parquet(OUT, index=False)
print(f"Wrote {OUT} windows={len(out)} vocab={len(vocab)}")

if __name__ == "__main__":
    main()

```

 Copy code

21.4 scripts/train/train_anomaly.py

python

```

import os
import mlflow
import pandas as pd
import numpy as np
import joblib
from sklearn.ensemble import IsolationForest

INP = "data/features/log_windows.parquet"
MODEL_OUT = "models/anomaly_model.pkl"

def main():
    os.makedirs("models", exist_ok=True)
    df = pd.read_parquet(INP)
    X = np.vstack(df["vec"].apply(lambda v: np.array(v, dtype=np.float32)))

    params = {"n_estimators": 200, "contamination": 0.02, "random_state": 42}
    model = IsolationForest(**params)
    model.fit(X)

```

```

# unsupervised: use avg anomaly score magnitude as a sanity metric
scores = -model.score_samples(X) # higher = more anomalous
metric = float(np.mean(scores))

mlflow.set_experiment("opspilot-anomaly")
with mlflow.start_run():
    mlflow.log_params(params)
    mlflow.log_metric("mean_anomaly_score", metric)
    joblib.dump(model, MODEL_OUT)
    mlflow.log_artifact(MODEL_OUT)

print(f"Saved model to {MODEL_OUT}, mean_anomaly_score={metric:.4f}")

if __name__ == "__main__":
    main()

```

 Copy code

21.5 src/opspilot/anomaly/infer.py

python

```

import joblib
import numpy as np
from typing import List, Dict, Any

class AnomalyScorer:
    def __init__(self, model_path: str):
        self.model = joblib.load(model_path)

    def score_vector(self, vec: List[float]) -> float:

```

```

X = np.array([vec], dtype=np.float32)
# IsolationForest: score_samples higher for normal; invert to anomaly
return float(-self.model.score_samples(X)[0])

def score_report(self, vec: List[float], top_templates: List[str]) -> Dict[str, Any]:
    score = self.score_vector(vec)
    return {"score": score, "top_templates": top_templates, "details": {}}

```

 Copy code

22. Agent Orchestration (LangGraph) + tools + prompts

22.1 src/opspilot/agent/prompts.py

python

```

SYSTEM = """You are OpsPilot, an incident response copilot.
Rules:
- Only recommend actions that are supported by retrieved evidence chunks.
- For every recommended action, include evidence doc_ids.
- If evidence is insufficient, say so and ask for specific missing info.
- Never provide destructive commands as direct instructions (e.g., rm -rf,
Output MUST be valid JSON matching the provided schema.
"""

USER_TEMPLATE = """Incident:
- incident_id: {incident_id}
- alert_title: {alert_title}
- service: {service}
- env: {environment}

```

```
- user_notes: {user_notes}
```

```
Anomaly report:
{anomaly_report}
```

```
Retrieved context (chunks):
{context}
```

```
Return JSON with:
```

```
incident_summary, probable_causes[], recommended_actions[], verification_status: str
"""
```

 Copy code

22.2 src/ospilot/agent/tools.py

python

```
from typing import Dict, Any, List
from ospilot.rag.retriever import HybridRetriever
from ospilot.anomaly.infer import AnomalyScorer

def tool_retrieve(retriever: HybridRetriever, query: str, top_k: int = 6)
    return retriever.search(query, top_k=top_k)

def tool_anomaly_score(scorer: AnomalyScorer, vec: List[float], top_templates: List[str])
    return scorer.score_report(vec, top_templates)

def tool_format_context(chunks: List[Dict[str, Any]]) -> str:
    # compact string; keep doc_id visible for evidence
    out = []
    for c in chunks:
```

```
out.append(f"[{c['doc_id']}] {c.get('title', '')} - {c.get('section')}
return "\n".join(out)
```

 Copy code

22.3 src/ospilot/agent/graph.py (LangGraph workflow skeleton)

python

```
from typing import TypedDict, List, Dict, Any, Optional
from langgraph.graph import StateGraph, END
import os, json
import httpx

from ospilot.agent.prompts import SYSTEM, USER_TEMPLATE
from ospilot.agent.safety import validate_grounded_actions

class AgentState(TypedDict, total=False):
    incident: Dict[str, Any]
    anomaly_report: Dict[str, Any]
    context_chunks: List[Dict[str, Any]]
    output: Dict[str, Any]
    trace: Dict[str, Any]

async def call_llm_json(prompt: str) -> Dict[str, Any]:
    provider = os.getenv("LLM_PROVIDER", "mock")
    if provider == "mock":
        # deterministic fallback for tests/dev without LLM
        return {
            "incident_summary": "Mock summary (LLM disabled).",
            "probable_causes": [{"cause": "Insufficient info", "confidence": 0.5},
            "recommended_actions": [],
```



```

        "verification_steps": [],
        "fallback_plan": [],
        "postmortem_draft_markdown": "## Postmortem\nLLM disabled.",
    }

# Ollama JSON mode is model-dependent; we enforce JSON by strict instr
host = os.getenv("OLLAMA_HOST", "http://localhost:11434")
model = os.getenv("OLLAMA_MODEL", "llama3.2:3b-instruct-q4_K_M")
payload = {
    "model": model,
    "prompt": prompt,
    "stream": False,
}

async with httpx.AsyncClient(timeout=60) as client:
    r = await client.post(f"{host}/api/generate", json=payload)
    r.raise_for_status()
    text = r.json().get("response", "")

# Try parse JSON from response (simple robust extraction)
start = text.find("{")
end = text.rfind("}")
if start == -1 or end == -1:
    raise ValueError("LLM did not return JSON.")
return json.loads(text[start:end+1])

def build_graph(retrieve_fn, anomaly_fn, format_context_fn):
    g = StateGraph(AgentState)

    def parse_request(state: AgentState) -> AgentState:
        state["trace"] = {"states": ["parse_request"]}
        return state

```

```
def anomaly_scan(state: AgentState) -> AgentState:
    state["trace"]["states"].append("anomaly_scan")
    # In real pipeline, compute vec + templates from logs; here assume
    vec = state["incident"].get("feature_vec", [])
    top_templates = state["incident"].get("top_templates", [])
    state["anomaly_report"] = anomaly_fn(vec, top_templates)
    return state

def retrieve_context(state: AgentState) -> AgentState:
    state["trace"]["states"].append("retrieve_context")
    q = f"{state['incident'].get('alert_title', '')} {state['incident']"
    state["context_chunks"] = retrieve_fn(q, top_k=int(os.getenv("RAG_
    return state

async def draft_response(state: AgentState) -> AgentState:
    state["trace"]["states"].append("draft_response")
    ctx = format_context_fn(state.get("context_chunks", []))
    prompt = SYSTEM + "\n\n" + USER_TEMPLATE.format(
        incident_id=state["incident"].get("incident_id"),
        alert_title=state["incident"].get("alert_title"),
        service=state["incident"].get("service"),
        environment=state["incident"].get("environment"),
        user_notes=state["incident"].get("user_notes"),
        anomaly_report=json.dumps(state.get("anomaly_report", {})), indi
        context=ctx
    )
    out = await call_llm_json(prompt)
    validate_grounded_actions(out, state.get("context_chunks", []))
    state["output"] = out
```

```

        return state

def done(state: AgentState) -> AgentState:
    state["trace"]["states"].append("done")
    return state

g.add_node("parse_request", parse_request)
g.add_node("anomaly_scan", anomaly_scan)
g.add_node("retrieve_context", retrieve_context)
g.add_node("draft_response", draft_response)
g.add_node("done", done)

g.set_entry_point("parse_request")
g.add_edge("parse_request", "anomaly_scan")
g.add_edge("anomaly_scan", "retrieve_context")
g.add_edge("retrieve_context", "draft_response")
g.add_edge("draft_response", "done")
g.add_edge("done", END)

return g.compile()

```

 Copy code

22.4 src/opspilot/agent/safety.py (evidence enforcement)

python

```

from typing import Dict, Any, List

def validate_grounded_actions(output: Dict[str, Any], context_chunks: List
    allowed_ids = {c["doc_id"] for c in context_chunks}
    actions = output.get("recommended_actions", []) or []

```

```
for a in actions:
    ev = a.get("evidence", []) or []
    # Require evidence if any action exists
    if not ev:
        raise ValueError("Ungrounded action: missing evidence doc_ids")
    # Evidence must refer to retrieved chunk IDs
    for doc_id in ev:
        if doc_id not in allowed_ids:
            raise ValueError(f"Ungrounded action: evidence doc_id not in allowed_ids")
```

[Copy code](#)

23. Wiring API routes to the agent

23.1 src/opspilot/api/routes/incident.py

python

```
from fastapi import APIRouter
from opspilot.api.schemas import IncidentRequest, IncidentAnalysisResponse
from opspilot.rag.index import FaissStore
from opspilot.rag.bm25 import BM25Index
from opspilot.rag.retriever import HybridRetriever
from opspilot.anomaly.infer import AnomalyScorer
from opspilot.agent.graph import build_graph
from opspilot.agent.tools import tool_format_context
import os
import pandas as pd
import numpy as np
```

```

router = APIRouter()

# --- naive singletons for demo; in prod use dependency injection ---
VECTOR_INDEX_PATH = os.getenv("VECTOR_INDEX_PATH", "./artifacts/vector_index")
EMBED_MODEL_NAME = os.getenv("EMBED_MODEL_NAME", "sentence-transformers/all-MiniLM-L6-v2")
ANOMALY_MODEL_PATH = os.getenv("ANOMALY_MODEL_PATH", "./models/anomaly_model")

faiss_store = FaissStore(VECTOR_INDEX_PATH, EMBED_MODEL_NAME).load()

# For BM25, we load meta.jsonl and index text quickly
meta = []
meta_file = os.path.join(VECTOR_INDEX_PATH, "meta.jsonl")
if os.path.exists(meta_file):
    with open(meta_file, "r", encoding="utf-8") as f:
        for line in f:
            meta.append(__import__("json").loads(line))
bm25 = BM25Index([m["text"] for m in meta], [m["doc_id"] for m in meta])

retriever = HybridRetriever(faiss_store, bm25, alpha=0.6)
scorer = AnomalyScorer(ANOMALY_MODEL_PATH)

graph = build_graph(
    retrieve_fn=lambda q, top_k: retriever.search(q, top_k=top_k),
    anomaly_fn=lambda vec, top_templates: scorer.score_report(vec, top_templates),
    format_context_fn=tool_format_context,
)

def _dummy_feature_vec(log_lines):
    # Replace with real feature extraction; this is a placeholder to keep the graph simple
    # Uses simple heuristics: count error words, length stats, etc.

```

```
text = "\n".join(log_lines).lower()
return [
    float(text.count("error")),
    float(text.count("exception")),
    float(len(log_lines)),
    float(sum(len(l) for l in log_lines) / max(1, len(log_lines))),
]

@router.post("/analyze", response_model=IncidentAnalysisResponse)
async def analyze(req: IncidentRequest):
    incident = req.model_dump()
    incident["feature_vec"] = _dummy_feature_vec(req.log_lines)
    incident["top_templates"] = [] # fill later from Drain3 parsing

    state = {"incident": incident}
    result = await graph.ainvoke(state)

    out = result.get("output", {})
    chunks = result.get("context_chunks", [])

    # map to response schema
    return IncidentAnalysisResponse(
        summary=out.get("incident_summary", ""),
        anomaly_report=AnomalyReport(**(result.get("anomaly_report") or {})),
        retrieved_context=[RetrievedChunk(**c) for c in chunks],
        actions=[RecommendedAction(**a) for a in out.get("recommended_actions", [])],
        verification_steps=out.get("verification_steps", []),
        fallback_plan=out.get("fallback_plan", []),
        postmortem_markdown=out.get("postmortem_draft_markdown", "")
```

 Copy code

```
        trace=result.get("trace", {}),  
    )
```

24. Streamlit UI (minimum that sells the story)

ui/streamlit_app.py

python

```
import os  
import streamlit as st  
import httpx  
import json  
  
API_BASE_URL = os.getenv("API_BASE_URL", "http://localhost:8000")  
  
st.set_page_config(page_title="OpsPilot", layout="wide")  
st.title("OpsPilot – Incident Response Copilot")  
  
with st.sidebar:  
    st.header("Incident Input")  
    incident_id = st.text_input("Incident ID", "INC-2026-00012")  
    alert_title = st.text_input("Alert Title", "NodeFilesystemSpaceFilling")  
    service = st.text_input("Service", "kubernetes")  
    environment = st.selectbox("Environment", ["prod", "staging", "dev"], index=0)  
    user_notes = st.text_area("Notes", "Pods failing scheduling; nodes show errors")  
    logs = st.text_area("Paste logs (one per line)", "ERROR disk full\nException in thread")  
    submit = st.button("Analyze Incident")
```

```
if submit:
    payload = {
        "incident_id": incident_id,
        "alert_title": alert_title,
        "service": service,
        "environment": environment,
        "user_notes": user_notes,
        "log_lines": [l for l in logs.splitlines() if l.strip()],
    }
    with st.spinner("Analyzing..."):
        r = httpx.post(f"{API_BASE_URL}/incident/analyze", json=payload, timeout=30)
        r.raise_for_status()
        data = r.json()

    col1, col2 = st.columns([2, 1])

    with col1:
        st.subheader("Triage Summary")
        st.write(data["summary"])

        st.subheader("Recommended Actions (evidence-backed)")
        for a in data["actions"]:
            st.markdown(f"- **({a['risk']})** {a['step']}")
            st.caption("Evidence: " + ", ".join(a.get("evidence", [])))

        st.subheader("Verification Steps")
        for v in data["verification_steps"]:
            st.markdown(f"- {v}")
```



```

st.subheader("Postmortem Draft")
st.markdown(data["postmortem_markdown"])

with col2:
    st.subheader("Anomaly Signal")
    st.metric("Anomaly Score", f"{data['anomaly_report']['score']:.3f}")

    st.subheader("Retrieved Context")
    for c in data["retrieved_context"]:
        with st.expander(f"{c['title']} – {c.get('section', '')} (score: {c['score']})"):
            st.write(c["text"])
            st.caption(c["doc_id"])

    st.subheader("Agent Trace")
    st.json(data.get("trace", {}))

st.divider()
st.subheader("Feedback")
helpful = st.radio("Was this helpful?", ["Yes", "No"], horizontal=True)
comment = st.text_area("Comment")
if st.button("Submit Feedback"):
    fb = {
        "incident_id": incident_id,
        "helpful": helpful == "Yes",
        "tags": [],
        "comment": comment,
    }
    httpx.post(f"{API_BASE_URL}/feedback", json=fb, timeout=20)
    st.success("Feedback saved.")

```

 Copy code

25. Observability (structured logs + Prometheus metrics)

25.1 src/opspilot/observability/logging.py

python

```
import logging
import os
import structlog

def configure_logging():
    level = os.getenv("LOG_LEVEL", "INFO").upper()
    logging.basicConfig(level=level)

    structlog.configure(
        processors=[
            structlog.processors.add_log_level,
            structlog.processors.TimeStamper(fmt="iso"),
            structlog.processors.JSONRenderer(),
        ],
        wrapper_class=structlog.make_filtering_bound_logger(getattr(logging,
    )
```

 Copy code

25.2 src/opspilot/observability/metrics.py

python

```
from prometheus_fastapi_instrumentator import Instrumentator
from fastapi import FastAPI
```

```
def instrument_app(app: FastAPI):  
    Instrumentator().instrument(app).expose(app, endpoint="/me1"  Copy code
```

26. Feedback endpoint + storage (SQLite/Postgres)

26.1 src/opspilot/storage/db.py

python

```
import os  
from sqlmodel import SQLModel, create_engine, Session  
  
DATABASE_URL = os.getenv("DATABASE_URL", "sqlite:///./opspilot.db")  
engine = create_engine(DATABASE_URL, echo=False)  
  
def init_db():  
    SQLModel.metadata.create_all(engine)  
  
def get_session():  
    with Session(engine) as session:  
        yield session
```

 Copy code

26.2 src/opspilot/storage/models.py

python

```
from sqlmodel import SQLModel, Field  
from typing import Optional
```

```

from datetime import datetime

class Feedback(SQLModel, table=True):
    id: Optional[int] = Field(default=None, primary_key=True)
    incident_id: str
    helpful: bool
    tags_json: str = "[]"
    comment: Optional[str] = None
    suggested_root_cause: Optional[str] = None
    created_at: datetime = Field(default_factory=datetime.utcnow)

```

 Copy code

26.3 src/opspilot/api/routes/feedback.py

python

```

from fastapi import APIRouter, Depends
from sqlmodel import Session
import json

from opspilot.api.schemas import FeedbackRequest
from opspilot.storage.db import get_session, init_db
from opspilot.storage.models import Feedback

router = APIRouter()

@router.post("")
def submit_feedback(req: FeedbackRequest, session: Session = Depends(get_session)):
    init_db()
    row = Feedback(
        incident_id=req.incident_id,

```

```
        helpful=req.helpful,  
        tags_json=json.dumps(req.tags),  
        comment=req.comment,  
        suggested_root_cause=req.suggested_root_cause,  
    )  
    session.add(row)  
    session.commit()  
    return {"status": "ok", "id": row.id}
```

 Copy code

27. Makefile targets (this is a “senior dev” green flag)

Makefile

makefile

```
.PHONY: setup lint test up down data features train index repro api ui
```

setup:

```
python -m pip install --upgrade pip  
pip install -e ".[api,ui]"  
pre-commit install || true
```

lint:

```
ruff check .  
mypy src/opspilot
```

test:

```
pytest -q
```

up:`docker compose up --build`**down:**`docker compose down -v`**data:**`python scripts/data/download_all.py`**features:**`python scripts/features/parse_logs.py``python scripts/features/build_features.py`**train:**`python scripts/train/train_anomaly.py`**index:**`python scripts/rag/build_index.py`**repro:**`dvc repro`**api:**`uvicorn opspilot.api.main:app --reload --port 8000`**ui:**`streamlit run ui/streamlit_app.py` Copy code

28. Build the runbook index script (key to RAG)

scripts/rag/build_index.py

python

```
import os
import glob
import hashlib

from opspilot.rag.index import FaissStore
from opspilot.rag.chunking import simple_md_chunk
import yaml

RUNBOOKS_DIR = "data/raw/runbooks"
OUT_INDEX = "artifacts/vector_index"

def read_md(path: str) -> str:
    with open(path, "r", encoding="utf-8", errors="ignore") as f:
        return f.read()

def main():
    os.makedirs(OUT_INDEX, exist_ok=True)

    embed_model = os.getenv("EMBED_MODEL_NAME", "sentence-transformers/all
    store = FaissStore(OUT_INDEX, embed_model)

    docs = []
    md_files = glob.glob(os.path.join(RUNBOOKS_DIR, "**/*.md"), recursive=True)
    for p in md_files:
        text = read_md(p)
        title = os.path.splitext(os.path.basename(p))[0]
```

```
chunks = simple_md_chunk(title=title, section="body", text=text, cl

for i, ch in enumerate(chunks):
    doc_id = f"runbook:{title}:{i}"
    docs.append({
        "doc_id": doc_id,
        "title": title,
        "section": "body",
        "text": ch,
        "source": "prometheus-operator-runbooks",
        "checksum": hashlib.sha256(ch.encode("utf-8")).hexdigest()
    })

if not docs:
    raise RuntimeError("No runbooks found. Ensure data/raw/runbooks has

store.build(docs)
print(f"Built FAISS index: {OUT_INDEX}, docs={len(docs)}")

if __name__ == "__main__":
    main()
```

 Copy code

29. Minimal tests that prove “production mindset”

29.1 tests/test_agent_safety.py

```
python
```



```
import pytest
from opspilot.agent.safety import validate_grounded_actions

def test_grounded_actions_require_evidence():
    ctx = [{"doc_id": "a"}, {"doc_id": "b"}]
    out = {"recommended_actions": [{"step": "Do X", "risk": "low", "evidence": "a"}]}
    with pytest.raises(ValueError):
        validate_grounded_actions(out, ctx)

def test_grounded_actions_must_reference_context_ids():
    ctx = [{"doc_id": "a"}]
    out = {"recommended_actions": [{"step": "Do X", "risk": "low", "evidence": "b"}]}
    with pytest.raises(ValueError):
        validate_grounded_actions(out, ctx)
```

 Copy code

29.2 tests/test_api_contract.py (smoke)

python

```
from fastapi.testclient import TestClient
from opspilot.api.main import create_app

def test_health():
    app = create_app()
    client = TestClient(app)
    r = client.get("/health")
    assert r.status_code == 200
    assert r.json()["status"] == "ok"
```

 Copy code

30. Step-by-step “first successful run” commands

bash

1) Create env + install

```
python -m venv .env
```

```
source .env/bin/activate
```

```
pip install -e ".[api,ui]"
```

2) Prepare data folders

```
mkdir -p data/raw/runbooks data/raw/loghub_subset
```

3) Download runbooks (example)

```
git clone https://github.com/prometheus-operator/runbooks.git data/raw/runbooks
```

4) Build RAG index

```
python scripts/rag/build_index.py
```

5) Start stack locally (fast path)

```
uvicorn opspilot.api.main:app --reload --port 8000
```

in a new terminal:

```
streamlit run ui/streamlit_app.py
```

 Copy code

Optional full stack:

bash

```
cp .env.example .env
```

```
docker compose up --build
```

 Copy code

If you type **“continue”**, I’ll add the remaining “no ambiguity” parts:

- DVC pipeline files (`dvc.yaml` , `params.yaml`) + MLflow logging for RAG eval
- Prefect flows: scheduled reindex, drift report (Evidently), retrain trigger
- A proper feature extractor that uses Drain3 online (not the dummy vec)
- CI/CD GitHub Actions (`ci.yaml` and docker build workflow)
- A killer `README.md` outline: architecture diagram, screenshots checklist, demo script, and interview talking points



continue

31. DVC pipeline + params (fully reproducible)

31.1 `params.yaml`

yaml

```
loghub:  
  dataset: hdfs  
  max_lines: 200000  
  
features:  
  window_size_lines: 200
```

```
vocab_size: 300

anomaly:
  model: isolation_forest
  n_estimators: 200
  contamination: 0.02
  random_state: 42

rag:
  chunk_size: 900
  chunk_overlap: 150
  top_k: 6
  hybrid_alpha: 0.6

eval:
  rag_gold_path: data/eval/rag_gold.jsonl
  top_k: 6
```

 Copy code

31.2 dvc.yaml

yaml

```
stages:
  download_data:
    cmd: python scripts/data/download_all.py
    outs:
      - data/raw

  parse_logs:
    cmd: python scripts/features/parse_logs.py
```

```
deps:
  - data/raw
  - drain3.ini
params:
  - loghub
outs:
  - data/processed/log_templates.parquet

build_features:
  cmd: python scripts/features/build_features.py
  deps:
    - data/processed/log_templates.parquet
  params:
    - features
  outs:
    - data/features/log_windows.parquet
    - artifacts/anomaly_vocab.json

train_anomaly_model:
  cmd: python scripts/train/train_anomaly.py
  deps:
    - data/features/log_windows.parquet
    - artifacts/anomaly_vocab.json
  params:
    - anomaly
  outs:
    - models/anomaly_model.pkl

build_rag_index:
  cmd: python scripts/rag/build_index.py
```

```
deps:
  - data/raw/runbooks

params:
  - rag

outs:
  - artifacts/vector_index

evaluate:
  cmd: python scripts/eval/run_eval.py
  deps:
    - models/anomaly_model.pkl
    - artifacts/vector_index
    - artifacts/anomaly_vocab.json
    - data/eval
  params:
    - eval
    - rag
  outs:
    - reports/eval.json
    - reports/rag_metrics.json
    - reports/anomaly_metrics.json
```

 Copy code

31.3 DVC commands (exact)

```
bash
```

```
dvc init
```

```
dvc add data/raw data/processed data/features artifacts models reports ||
```

```
git add .
```

 Copy code

```
git commit -m "Add DVC pipeline skeleton"
dvc repro
```

[Copy code](#)

Tip: Keep `data/raw` minimal (subset) so reviewers can actually run it.

32. RAG evaluation harness (no paid LLM judges)

32.1 Gold dataset format: `data/eval/rag_gold.jsonl`

Each line maps a query to expected doc_ids (runbook chunk IDs).

jsonl

```
{"query":"NodeFilesystemSpaceFillingUp disk pressure","expected_doc_ids":[
{"query":"KubePodCrashLooping application crash","expected_doc_ids":["runb
```

[Copy code](#)

You'll create 20–50 lines manually. This alone makes your project “serious”.

32.2 `scripts/eval/run_eval.py` (RAG metrics + anomaly sanity)

python

```
import os, json
import numpy as np
import joblib
from typing import List, Dict, Any
from opspilot.rag.index import FaissStore
```

```
RAG_INDEX = os.getenv("VECTOR_INDEX_PATH", "./artifacts/vector_index")
EMBED_MODEL_NAME = os.getenv("EMBED_MODEL_NAME", "sentence-transformers/all-mpnet-base-v2")
GOLD = "data/eval/rag_gold.jsonl"

def recall_at_k(retrieved: List[str], expected: List[str], k: int) -> float:
    r = set(retrieved[:k])
    e = set(expected)
    return 1.0 if len(r & e) > 0 else 0.0

def mrr(retrieved: List[str], expected: List[str]) -> float:
    e = set(expected)
    for i, doc_id in enumerate(retrieved, start=1):
        if doc_id in e:
            return 1.0 / i
    return 0.0

def main():
    store = FaissStore(RAG_INDEX, EMBED_MODEL_NAME).load()
    k = int(os.getenv("RAG_TOP_K", "6"))

    rows = []
    with open(GOLD, "r", encoding="utf-8") as f:
        for line in f:
            rows.append(json.loads(line))

    recs, mrrs = [], []
    per_query = []
    for r in rows:
        hits = store.search(r["query"], top_k=k)
        retrieved_ids = [h[0]["doc_id"] for h in hits]
```



```

recs.append(recall_at_k(retrieved_ids, r["expected_doc_ids"], k))
mrrs.append(mrr(retrieved_ids, r["expected_doc_ids"]))
per_query.append({
    "query": r["query"],
    "retrieved": retrieved_ids,
    "expected": r["expected_doc_ids"],
})

metrics = {
    "recall_at_k": float(np.mean(recs)) if recs else 0.0,
    "mrr": float(np.mean(mrrs)) if mrrs else 0.0,
    "n_queries": len(rows),
    "k": k,
}

os.makedirs("reports", exist_ok=True)
with open("reports/rag_metrics.json", "w", encoding="utf-8") as f:
    json.dump(metrics, f, indent=2)
with open("reports/eval.json", "w", encoding="utf-8") as f:
    json.dump({"rag": metrics, "per_query": per_query[:50]}, f, indent=2)

print(json.dumps(metrics, indent=2))

if __name__ == "__main__":
    main()

```

 Copy code

32.3 Log metrics to MLflow (add to `run_eval.py` easily)

Add:

python

```
import mlflow
mlflow.set_experiment("opspilot-eval")
with mlflow.start_run():
    mlflow.log_metrics(metrics)
    mlflow.log_artifact("reports/rag_metrics.json")
```

 Copy code

33. Replace dummy feature vector with a real online Drain3 + vocab pipeline

You already have offline template mining; now you'll use the learned vocab during API inference.

33.1 Save vocab during feature build

Update `scripts/features/build_features.py` to write `artifacts/anomaly_vocab.json`:

python

```
import json
# after vocab computed
os.makedirs("artifacts", exist_ok=True)
with open("artifacts/anomaly_vocab.json", "w") as f:
    json.dump(vocab, f)
```

 Copy code

33.2 `src/opspilot/anomaly/features.py` (online feature extraction)

python

```
import json
import numpy as np
from collections import Counter
from typing import List
from drain3 import TemplateMiner
from drain3.template_miner_config import TemplateMinerConfig

class OnlineLogFeaturizer:
    """
    Online Drain3 template mining + fixed vocab vectorization.
    """
    def __init__(self, vocab_path: str = "./artifacts/anomaly_vocab.json",
                 with_open(vocab_path, "r") as f:
        self.vocab = json.load(f)
        self.vocab_set = set(self.vocab)

        cfg = TemplateMinerConfig()
        cfg.load(drain3_ini)
        self.miner = TemplateMiner(cfg)

    def featurize(self, log_lines: List[str]) -> tuple[list[float], list[str]]:
        cluster_ids = []
        templates = []
        for line in log_lines:
            res = self.miner.add_log_message(line.strip())
            cid = res["cluster_id"]
            tpl = res["template_mined"]
            cluster_ids.append(cid)
            templates.append(tpl)
```

```

cnt = Counter([c for c in cluster_ids if c in self.vocab_set])
vec = [float(cnt.get(c, 0)) for c in self.vocab]

# top templates by frequency (human friendly)
top = Counter(templates).most_common(5)
top_templates = [t for t, _ in top]
return vec, top_templates

```

 Copy code

33.3 Update `incident.py` route to use real featurizer

Replace `_dummy_feature_vec` logic with:

python

```

from opspilot.anomaly.features import OnlineLogFeaturizer
featurizer = OnlineLogFeaturizer("./artifacts/anomaly_vocab.json", "./drain

# in analyze():
vec, top_templates = featurizer.featurize(req.log_lines)
incident["feature_vec"] = vec
incident["top_templates"] = top_templates

```

 Copy code

Now your “anomaly signal” is real and consistent with training vocab.

34. Prefect flows: reindex, drift, retrain trigger (impressive + buildable)

34.1 `src/ospilot/workflows/prefect_flows.py`

python

```
from prefect import flow, task
import subprocess
import os
from datetime import datetime

@task(retries=2, retry_delay_seconds=10)
def run(cmd: str):
    print(f"Running: {cmd}")
    subprocess.check_call(cmd, shell=True)

@flow(name="ospilot-reindex-runbooks")
def reindex_runbooks():
    run("python scripts/rag/build_index.py")
    # optional: log build timestamp
    os.makedirs("reports", exist_ok=True)
    with open("reports/reindex.log", "a") as f:
        f.write(f"{datetime.utcnow().isoformat()} reindex ok\n")

@flow(name="ospilot-train-anomaly")
def train_anomaly():
    run("python scripts/train/train_anomaly.py")

@flow(name="ospilot-nightly")
def nightly():
    # lightweight nightly pipeline
    run("dvc repro evaluate")
```

```
if __name__ == "__main__":  
    nightly()
```

34.2 Drift report with Evidently: `src/opspilot/workflows/drift.py`

This monitors distribution drift in anomaly vectors and anomaly scores.

python

```
import os  
import json  
import pandas as pd  
import numpy as np  
from evidently.report import Report  
from evidently.metric_preset import DataDriftPreset  
  
BASELINE = "data/features/log_windows.parquet"  
CURRENT = "data/features/log_windows.parquet" # replace with latest daily  
OUT = "reports/drift_report.html"  
  
def to_matrix(df):  
    return np.vstack(df["vec"].apply(lambda v: np.array(v, dtype=np.float32)))  
  
def main():  
    os.makedirs("reports", exist_ok=True)  
    base = pd.read_parquet(BASELINE).head(500)  
    cur = pd.read_parquet(CURRENT).tail(500)  
  
    # Evidently expects a table. Expand vectors to columns.  
    Xb = to_matrix(base)  
    Xc = to_matrix(cur)
```

```

cols = [f"f{i}" for i in range(Xb.shape[1])]
base_df = pd.DataFrame(Xb, columns=cols)
cur_df = pd.DataFrame(Xc, columns=cols)

report = Report(metrics=[DataDriftPreset()])
report.run(reference_data=base_df, current_data=cur_df)
report.save_html(OUT)
print(f"Wrote {OUT}")

if __name__ == "__main__":
    main()

```

 Copy code

34.3 Run Prefect locally (free)

bash

```

prefect server start
# In another terminal:
prefect worker start --pool default-agent-pool
# Trigger flow:
python -c "from opspilot.workflows.prefect_flows import nightly; nightly()"

```

 Copy code

35. CI/CD: GitHub Actions (realistic, production-style)

35.1 .github/workflows/ci.yml

yaml

```
name: CI

on:
  pull_request:
  push:
    branches: [ "main", "dev" ]

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      - name: Set up Python
        uses: actions/setup-python@v5
        with:
          python-version: "3.11"

      - name: Install
        run: |
          python -m pip install --upgrade pip
          pip install -e ".[api,ui]"
          pip install pytest ruff mypy

      - name: Lint
        run: |
          ruff check .
          mypy src/opspilot || true # if you are still ramping mypy, remove
```



```
- name: Tests
  run: pytest -q
```

[Copy code](#)

35.2 `.github/workflows/docker-build.yml`

yaml

```
name: Docker Build

on:
  push:
    branches: ["main"]
    workflow_dispatch:

jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Build API image
        run: docker build -f docker/api.Dockerfile -t opspilot-api:latest
      - name: Build UI image
        run: docker build -f docker/ui.Dockerfile -t opspilot-ui:latest
```

[Copy code](#)

35.3 (Optional) Publish images to GHCR: `.github/workflows/release.yml`

This is optional but looks premium.

yaml

```
name: Release

on:
  push:
    tags:
      - "v*.*.*"

jobs:
  push:
    runs-on: ubuntu-latest
    permissions:
      contents: read
      packages: write
    steps:
      - uses: actions/checkout@v4
      - name: Login to GHCR
        run: echo "${{ secrets.GITHUB_TOKEN }}" | docker login ghcr.io -u ${{ github.repository_owner }}

      - name: Build & push API
        run: |
          IMAGE=ghcr.io/${{ github.repository_owner }}/opspilot-api:${{ github.ref_name }}
          docker build -f docker/api.Dockerfile -t $IMAGE .
          docker push $IMAGE

      - name: Build & push UI
        run: |
          IMAGE=ghcr.io/${{ github.repository_owner }}/opspilot-ui:${{ github.ref_name }}
          docker build -f docker/ui.Dockerfile -t $IMAGE .
          docker push $IMAGE
```

 Copy code

```
docker build -f docker/ui.Dockerfile -t $IMAGE .  
docker push $IMAGE
```

36. Production README outline (what top reviewers expect)

README.md (structure)

1. Project overview

- 2–3 lines
- key capabilities: anomaly detection + RAG + tool-using agent

2. Live demo (if deployed) + screenshots/GIF

3. Architecture

- the text diagram you already have
- explain offline/online separation

4. Features

- grounded actions with evidence
- hybrid retrieval
- Drift monitoring + feedback loop
- Dockerized full stack

5. Quickstart (local)

```
bash
```

```
cp .env.example .env  
docker compose up --build
```

 Copy code

6. Reproducible training

```
bash
```

```
dvc repro  
mlflow ui ...
```

[Copy code](#)

7. API docs

- `/docs` screenshot
- example curl:

```
bash
```

```
curl -X POST http://localhost:8000/incident/analyze -H "Content-Type: application/json"
```

[Copy code](#)

8. Evaluation

- Retrieval metrics (`reports/rag_metrics.json`)
- Anomaly metrics (`reports/anomaly_metrics.json`)

9. Observability

- Prometheus metrics endpoint
- Grafana dashboard import instructions

10. Security

- prompt injection mitigation rules
- evidence allow-list enforcement

11. Roadmap

- async jobs
- multi-LLM routing
- RBAC

12. Project demo script (what you'll say in interviews)

- "Given alert X, here's how system executes"

- Show trace, show evidence, show metrics

37. “Demo script” for interviews (what to literally do on screen)

1. Start stack:

```
bash
```

```
docker compose up --build
```

 Copy code

2. Open:

- UI: localhost:8501
- API docs: localhost:8000/docs
- Grafana: localhost:3000

3. Paste an incident + logs

4. Show:

- anomaly score
- retrieved runbook chunks
- recommended actions with evidence doc_ids
- agent trace states

5. Submit feedback

6. Show DB row inserted (optional)

7. Show Prometheus metrics endpoint and a Grafana panel

This makes you look like someone who has shipped systems, not notebooks.

38. “No ambiguity” finishing touches (small but high-signal)

38.1 Add examples folder (reviewers love this)

pgsql

examples/

- |— incident_node_disk.json
- |— incident_crashloop.json
- |— rag_gold_seed.jsonl

 Copy code

38.2 Add `pre-commit` config (optional but strong): `.pre-commit-config.yaml`

yaml

repos:

- repo: <https://github.com/astral-sh/ruff-pre-commit>
rev: v0.5.7
hooks:
 - id: ruff
 - id: ruff-format
- repo: <https://github.com/pre-commit/mirrors-mypy>
rev: v1.11.1
hooks:
 - id: mypy

 Copy code

39. What to build next (highest ROI add-ons)

If you want “top-tier product company” impact, implement these next:

1. Async reindex endpoint:

- `/admin/reindex` triggers Prefect flow; UI shows progress

2. Multi-LLM routing

- Use small model for planning; bigger for final narrative

3. RAG hybrid docstore fix

- Right now BM25-only hits don't have meta; add a `DocStore` keyed by `doc_id` so both retrieval paths are complete

4. Guardrails

- block suspicious instructions from context ("prompt injection" patterns)
- strip markdown code blocks from retrieved runbook chunks during prompt assembly

If you type **"continue"**, I'll finish with:

- a full `DocStore` abstraction + complete hybrid retrieval (no missing meta)
- a proper `/rag/search` and `/anomaly/score` endpoints
- a minimal RBAC JWT auth implementation
- a complete "Interview trade-offs pack" (10+ deep trade-offs with crisp answers)
- final polished resume bullets with quantified metrics you can honestly claim (based on your eval outputs)



continue

40. Complete docstore + “no missing meta” hybrid retrieval (production fix)

Right now BM25 hits may lack metadata. Fix by introducing a **DocStore** that is the single source of truth for documents.

40.1 src/opspilot/rag/docstore.py

python

```
import os, json
from typing import Dict, Any, Iterable, Optional, List

class DocStore:
    """
    Simple local docstore keyed by doc_id.
    Backed by JSONL file produced during index build.
    """
    def __init__(self, meta_jsonl_path: str):
        self.meta_jsonl_path = meta_jsonl_path
        self._docs: Dict[str, Dict[str, Any]] = {}

    def load(self) -> "DocStore":
        self._docs = {}
        if not os.path.exists(self.meta_jsonl_path):
            return self
        with open(self.meta_jsonl_path, "r", encoding="utf-8") as f:
            for line in f:
```



```

        d = json.loads(line)
        self._docs[d["doc_id"]] = d
    return self

def get(self, doc_id: str) -> Optional[Dict[str, Any]]:
    return self._docs.get(doc_id)

def all_docs(self) -> List[Dict[str, Any]]:
    return list(self._docs.values())

def all_texts_and_ids(self) -> tuple[list[str], list[str]]:
    docs = self.all_docs()
    return [d["text"] for d in docs], [d["doc_id"] for d in docs]
```

 Copy code

40.2 Update FAISS store to rely on docstore ordering

Modify `FaissStore` to load meta via docstore:

```
python

# in opspilot/rag/index.py
from opspilot.rag.docstore import DocStore

class FaissStore:
    def __init__(...):
        ...
        self.docstore = DocStore(self.meta_file)

    def load(self):
        ...
        self.docstore.load()
```

```
# Maintain a stable list of docs to map idx->doc
self._meta = self.docstore.all_docs()
return self
```

 Copy code

40.3 src/opspilot/rag/retriever.py (final hybrid retriever)

python

```
from typing import List, Dict, Any, Tuple
from opspilot.rag.index import FaissStore
from opspilot.rag.bm25 import BM25Index
from opspilot.rag.docstore import DocStore

class HybridRetriever:
    def __init__(self, faiss_store: FaissStore, docstore: DocStore, alpha:
        self.faiss = faiss_store
        self.docstore = docstore
        texts, ids = self.docstore.all_texts_and_ids()
        self.bm25 = BM25Index(texts, ids)
        self.alpha = alpha

    def _normalize(self, d: Dict[str, float]) -> Dict[str, float]:
        if not d:
            return {}
        mx = max(d.values()) or 1.0
        return {k: v / mx for k, v in d.items()}

    def search(self, query: str, top_k: int = 6) -> List[Dict[str, Any]]:
        vec_hits = self.faiss.search(query, top_k=top_k * 4)
        bm_hits = self.bm25.search(query, top_k=top_k * 4)
```

```
vec_scores = {meta["doc_id"]: score for meta, score in vec_hits}
bm_scores = {doc_id: float(score) for doc_id, score in bm_hits}

nvec = self._normalize(vec_scores)
nbm = self._normalize(bm_scores)
all_ids = set(nvec) | set(nbm)

fused: List[Tuple[str, float]] = []
for doc_id in all_ids:
    s = self.alpha * nvec.get(doc_id, 0.0) + (1 - self.alpha) * nbm.get(doc_id, 0.0)
    fused.append((doc_id, s))

fused.sort(key=lambda x: x[1], reverse=True)
results = []
for doc_id, score in fused[:top_k]:
    doc = self.docstore.get(doc_id)
    if not doc:
        continue
    results.append({
        "doc_id": doc_id,
        "title": doc.get("title", ""),
        "section": doc.get("section", ""),
        "text": doc["text"],
        "score": float(score),
        "source": doc.get("source", ""),
    })
return results
```

 Copy code

40.4 Update `incident.py` wiring

Replace earlier BM25 initialization with:

python

```
from opspilot.rag.docstore import DocStore
docstore = DocStore(os.path.join(VECTOR_INDEX_PATH, "meta.jsonl")).load()
retriever = HybridRetriever(faiss_store, docstore, alpha=float(os.getenv("I
```

 Copy code

Now hybrid retrieval is fully correct.

41. Add `/rag/search` and `/anomaly/score` endpoints (clean API surface)

41.1 `src/opspilot/api/routes/rag.py`

python

```
from fastapi import APIRouter
from opspilot.api.schemas import RagSearchRequest, RetrievedChunk
from opspilot.rag.index import FaissStore
from opspilot.rag.docstore import DocStore
from opspilot.rag.retriever import HybridRetriever
import os

router = APIRouter()

VECTOR_INDEX_PATH = os.getenv("VECTOR_INDEX_PATH", "./artifacts/vector_index")
```

```

EMBED_MODEL_NAME = os.getenv("EMBED_MODEL_NAME", "sentence-transformers/all
HYBRID_ALPHA = float(os.getenv("HYBRID_ALPHA", "0.6"))

faiss_store = FaissStore(VECTOR_INDEX_PATH, EMBED_MODEL_NAME).load()
docstore = DocStore(os.path.join(VECTOR_INDEX_PATH, "meta.jsonl")).load()
retriever = HybridRetriever(faiss_store, docstore, alpha=HYBRID_ALPHA)

@router.post("/search", response_model=list[RetrievedChunk])
def rag_search(req: RagSearchRequest):
    hits = retriever.search(req.query, top_k=req.top_k)
    return [RetrievedChunk(**h) for h in hits]

```

 Copy code

41.2 src/opspilot/api/routes/anomaly.py

python

```

from fastapi import APIRouter
from opspilot.api.schemas import AnomalyScoreRequest, AnomalyReport
from opspilot.anomaly.features import OnlineLogFeaturizer
from opspilot.anomaly.infer import AnomalyScorer
import os

router = APIRouter()

ANOMALY_MODEL_PATH = os.getenv("ANOMALY_MODEL_PATH", "./models/anomaly_model")
VOCAB_PATH = os.getenv("ANOMALY_VOCAB_PATH", "./artifacts/anomaly_vocab.json")

scorer = AnomalyScorer(ANOMALY_MODEL_PATH)
featurizer = OnlineLogFeaturizer(VOCAB_PATH, "./drain3.ini")

```

```
@router.post("/score", response_model=AnomalyReport)
def score(req: AnomalyScoreRequest):
    vec, top_templates = featurizer.featurize(req.log_lines)
    rep = scorer.score_report(vec, top_templates)
    return AnomalyReport(**rep)
```

 Copy code

42. Minimal JWT auth + RBAC (optional but “senior”)

42.1 Add dependency: pyjwt

Add to `pyproject.toml`:

toml

```
PyJWT = "^2.9.0"
passlib = {extras=["bcrypt"], version="^1.7.4"}
```

 Copy code

42.2 `src/opspilot/api/deps.py`

python

```
import os
import jwt
from fastapi import Depends, HTTPException
from fastapi.security import HTTPAuthorizationCredentials, HTTPBearer

security = HTTPBearer(auto_error=False)

def _enabled() -> bool:
```

```
return os.getenv("ENABLE_AUTH", "false").lower() == "true"

def get_current_user(creds: HTTPAuthorizationCredentials = Depends(security),
                    if not _enabled():
                        return {"sub": "local-dev", "role": "admin"}

                    if creds is None:
                        raise HTTPException(status_code=401, detail="Missing token")

                    secret = os.getenv("JWT_SECRET", "change_me")
                    try:
                        payload = jwt.decode(creds.credentials, secret, algorithms=["HS256"])
                    except Exception:
                        raise HTTPException(status_code=401, detail="Invalid token")
                    return payload

def require_role(*roles: str):
    def dep(user=Depends(get_current_user)):
        if user.get("role") not in roles:
            raise HTTPException(status_code=403, detail="Forbidden")
        return user
    return dep
```

 Copy code

42.3 Protect admin endpoints (example)

Create `src/opspilot/api/routes/admin.py`:

```
python
```

```

from fastapi import APIRouter, Depends
from opspilot.api.deps import require_role
import subprocess

router = APIRouter()

@router.post("/reindex", dependencies=[Depends(require_role("admin"))])
def reindex():
    subprocess.check_call("python scripts/rag/build_index.py", shell=True)
    return {"status": "ok"}

@router.post("/retrain", dependencies=[Depends(require_role("admin"))])
def retrain():
    subprocess.check_call("python scripts/train/train_anomaly.py", shell=True)
    return {"status": "ok"}

```

 Copy code

Then include it in `main.py` with prefix `/admin`.

For a portfolio demo you can keep `ENABLE_AUTH=false` by default and show how to enable it.

43. “Interview trade-offs pack” (crisp senior answers)

Use these as rehearsed mini-speeches.

1. Why hybrid retrieval?

Vector search captures semantic similarity; BM25 nails exact alert names, metric labels, and error codes. Hybrid improves Recall@K especially on short queries.

2. Why FAISS local instead of managed vector DB?

Constraint: free + laptop-friendly. FAISS gives production-like performance without external dependencies. I abstracted vector store so swapping to Qdrant is trivial later.

3. How do you prevent hallucinations?

Hard requirement: every action must cite evidence doc_ids from retrieved context. I enforce this in code (`validate_grounded_actions`) so it's not just "prompting".

4. Why use an agent at all?

Because the workflow is multi-step and tool-based: anomaly scoring, retrieval, structured synthesis, postmortem drafting. LangGraph provides a deterministic state machine and debuggable trace.

5. How do you evaluate GenAI without paid judges?

I use retrieval metrics (Recall@K, MRR), groundedness checks (% actions with evidence), and a human feedback UI. This is realistic for teams without paid eval infra.

6. CPU-only model choice trade-off

Small quantized LLM reduces latency and cost; I compensate by emphasizing retrieval grounding and structured templates. The LLM is used for synthesis, not "knowledge".

7. Why DVC + MLflow together?

DVC versions data and pipelines; MLflow tracks experiments/metrics and acts as a registry. Together they provide end-to-end reproducibility.

8. What about prompt injection from runbooks?

Treat runbook text as untrusted. Strip suspicious patterns, never execute commands, and ensure recommendations are allow-listed and evidence-bound.

9. How do you handle drift?

Monitor template distribution and anomaly score distribution. If drift exceeds thresholds, trigger retraining pipeline; keep old model as fallback.

10. How would you scale this?

Async ingestion + reindexing, Redis cache, background job queue, and a dedicated vector DB. Horizontally scale API; keep indexes versioned and hot-swappable.

44. Final “quantified” resume bullets you can honestly claim

Once you run `scripts/eval/run_eval.py`, you’ll have actual metrics.

Template bullets you can fill with your numbers:

- Improved runbook retrieval **Recall@6 from X → Y** by implementing **hybrid BM25 + embedding retrieval** and tuning fusion weight (alpha).
- Reduced “ungrounded recommendations” to **0%** by enforcing **evidence-linked actions** via programmatic validation.
- Built reproducible pipelines with **DVC** and tracked **N+ experiments** in **MLflow**, enabling one-command rebuild and evaluation (`dvc repro`).
- Instrumented FastAPI with **Prometheus** histograms and Grafana dashboards to monitor **p95 latency**, error rates, and tool failures.

45. Final checklist: what your GitHub must contain to be “flagship”

-  Architecture diagram + offline/online flow
-  Reproducible `dvc repro` pipeline
-  MLflow tracking screenshots

-  Docker Compose full stack
-  Observability (metrics + dashboard)
-  Retrieval eval dataset + metrics
-  Agent trace + groundedness enforcement
-  Tests + CI passing badge
-  Demo GIF + “interview demo script”

46. “If I were hiring, what I’d ask you to improve” (great to mention proactively)

- Add correlation across multiple alerts (incident clustering)
- Add event timeline reconstruction from logs
- Add safe remediation playbooks (allow-list)
- Improve reranking (lightweight cross-encoder optional)
- Add Qdrant swap (docker) behind interface
- Add multi-tenant RBAC (org/team)

Mentioning these shows roadmap thinking.

If you want **one last “continue”**, I can provide:

- a complete `download_all.py` that pulls Loghub subset + runbooks cleanly
- a Grafana dashboard JSON (importable)
- a “One-page system design doc” (`docs/system_design.md`) that looks like an internal RFC



[continue](#)

47. Data download scripts (clean + reproducible)

47.1 `scripts/data/download_all.py`

python

```
import os
import shutil
import subprocess
from pathlib import Path

RAW = Path("data/raw")
EXTERNAL = Path("external")

LOGHUB_REPO = "https://github.com/logpai/loghub.git"
RUNBOOKS_REPO = "https://github.com/prometheus-operator/runbooks.git"

def run(cmd: str):
    print(f"> {cmd}")
    subprocess.check_call(cmd, shell=True)

def ensure_repo(url: str, dst: Path):
    if dst.exists() and (dst / ".git").exists():
        run(f"git -C {dst} pull --ff-only")
```

```
else:
    dst.parent.mkdir(parents=True, exist_ok=True)
    run(f"git clone --depth 1 {url} {dst}")

def copy_loghub_subset(loghub_path: Path):
    """
    Copies only a small subset to keep repo runnable on laptops.
    Adjust paths depending on Loghub structure.
    """
    # Common file locations in loghub: data/ or specific dataset folder.
    # We'll attempt a few known patterns.
    candidates = [
        loghub_path / "HDFS" / "HDFS.log",
        loghub_path / "data" / "HDFS" / "HDFS.log",
        loghub_path / "data" / "HDFS" / "HDFS_2k.log",
    ]
    src = None
    for c in candidates:
        if c.exists():
            src = c
            break

    if src is None:
        # fallback: search for any HDFS.log
        hits = list(loghub_path.rglob("HDFS.log"))
        if hits:
            src = hits[0]

    if src is None:
        raise FileNotFoundError("Could not locate HDFS log in Loghub. Inspi
```

```
out_dir = RAW / "loghub_subset" / "hdfs"
out_dir.mkdir(parents=True, exist_ok=True)
dst = out_dir / "HDFS.log"
shutil.copyfile(src, dst)
print(f"Copied {src} -> {dst}")

def copy_runbooks(runbooks_path: Path):
    out_dir = RAW / "runbooks"
    if out_dir.exists():
        shutil.rmtree(out_dir)
    shutil.copytree(runbooks_path, out_dir, ignore=shutil.ignore_patterns('
print(f"Copied runbooks -> {out_dir}")

def main():
    RAW.mkdir(parents=True, exist_ok=True)
    EXTERNAL.mkdir(parents=True, exist_ok=True)

    loghub_path = EXTERNAL / "loghub"
    runbooks_path = EXTERNAL / "runbooks"

    ensure_repo(LOGHUB_REPO, loghub_path)
    ensure_repo(RUNBOOKS_REPO, runbooks_path)

    copy_loghub_subset(loghub_path)
    copy_runbooks(runbooks_path)

    print("Download complete.")
```

```
if __name__ == "__main__":  
    main()
```

47.2 Verify downloads

bash

```
python scripts/data/download_all.py  
ls -R data/raw
```

 Copy code

48. Grafana dashboard (importable JSON)

This is a **minimal** dashboard you can import into Grafana to show:

- request rate
- latency (p50/p95)
- error rate (5xx)

48.1 docker/grafana_dashboard_opspilot.json

json

```
{  
  "annotations": {  
    "list": [  
      {  
        "builtIn": 1,  
        "datasource": "-- Grafana --",  
        "enable": true,
```

```
        "hide": true,
        "iconColor": "rgba(0, 211, 255, 1)",
        "name": "Annotations & Alerts",
        "type": "dashboard"
    }
]
},
"editable": true,
"fiscalYearStartMonth": 0,
"graphTooltip": 0,
"id": null,
"links": [],
"liveNow": false,
"panels": [
    {
        "datasource": { "type": "prometheus", "uid": "PROMETHEUS_DS_UID" },
        "fieldConfig": { "defaults": {}, "overrides": [] },
        "gridPos": { "h": 8, "w": 12, "x": 0, "y": 0 },
        "id": 1,
        "options": { "legend": { "displayMode": "list", "placement": "bottom" },
        "targets": [
            {
                "expr": "sum(rate(http_requests_total{job=\"opspilot-api\"}[1m]))",
                "legendFormat": "RPS",
                "refId": "A"
            }
        ],
        "title": "API Requests / sec",
        "type": "timeseries"
    },
],
```



```

{
  "datasource": { "type": "prometheus", "uid": "PROMETHEUS_DS_UID" },
  "fieldConfig": { "defaults": {}, "overrides": [] },
  "gridPos": { "h": 8, "w": 12, "x": 12, "y": 0 },
  "id": 2,
  "options": { "legend": { "displayMode": "list", "placement": "bottom" },
  "targets": [
    {
      "expr": "histogram_quantile(0.50, sum by (le) (rate(http_request_
      \"legendFormat\": \"p50\",
      \"refId\": \"A\"
    },
    {
      "expr": "histogram_quantile(0.95, sum by (le) (rate(http_request_
      \"legendFormat\": \"p95\",
      \"refId\": \"B\"
    }
  ],
  "title": "API Latency (seconds)",
  "type": "timeseries"
},
{
  "datasource": { "type": "prometheus", "uid": "PROMETHEUS_DS_UID" },
  "fieldConfig": { "defaults": {}, "overrides": [] },
  "gridPos": { "h": 8, "w": 12, "x": 0, "y": 8 },
  "id": 3,
  "options": { "legend": { "displayMode": "list", "placement": "bottom" },
  "targets": [
    {
      "expr": "sum(rate(http_requests_total{job=\"opspilot-api\", statu:

```

```
        "legendFormat": "5xx / sec",
        "refId": "A"
    }
],
"title": "API 5xx Errors",
"type": "timeseries"
},
{
    "datasource": { "type": "prometheus", "uid": "PROMETHEUS_DS_UID" },
    "fieldConfig": { "defaults": {}, "overrides": [] },
    "gridPos": { "h": 8, "w": 12, "x": 12, "y": 8 },
    "id": 4,
    "options": { "legend": { "displayMode": "list", "placement": "bottom" },
    "targets": [
        {
            "expr": "sum(rate(http_requests_total{job=\"opspilot-api\"}[5m])",
            "legendFormat": "{{handler}}",
            "refId": "A"
        }
    ],
    "title": "Requests by Endpoint",
    "type": "timeseries"
}
],
"refresh": "10s",
"schemaVersion": 39,
"style": "dark",
"tags": ["opspilot", "fastapi", "prometheus"],
"templating": { "list": [] },
"time": { "from": "now-30m", "to": "now" },
```

```
"timepicker": {},  
"timezone": "",  
"title": "OpsPilot - API Overview",  
"uid": "opspilot-api",  
"version": 1  
}
```

 Copy code

48.2 Hook Grafana datasource UID quickly

Grafana assigns a datasource UID when you add Prometheus. You have two easy options:

Option A (manual):

1. Grafana → Connections → Data sources → Prometheus → note its UID
2. Replace `PROMETHEUS_DS_UID` in JSON with that UID
3. Dashboards → Import → upload JSON

Option B (fast demo):

In Grafana import UI, you can select the datasource during import; Grafana will map it even if UID mismatches (depends on version).

49. One-page System Design RFC (interview-ready doc)

Create: `docs/system_design.md`

md

OpsPilot System Design (RFC)

Problem

On-call engineers waste time correlating alerts with logs and runbooks. MT

- weak search over docs/runbooks
- inconsistent triage workflow
- inability to reuse learnings from past incidents

Goals

- Provide fast incident triage: anomaly signal + runbook retrieval + structured recommendations
- Ground all recommendations in evidence
- Reproducible pipelines (data, training, indexing)
- Production observability (latency, errors, drift)
- Run locally without GPU or paid APIs

Non-Goals

- Automated remediation execution (dangerous); OpsPilot only recommends steps
- Full enterprise integrations (PagerDuty/Jira) unless optional

Architecture

Components:

- FastAPI: stable API for UI + automation
- Streamlit UI: incident console + evidence viewer + feedback capture
- Anomaly Service: Drain3 + IsolationForest scoring
- RAG: FAISS vector index + BM25 hybrid retrieval over runbooks
- Agent Orchestrator: LangGraph workflow producing structured JSON
- Stores:
 - SQL (SQLite/Postgres): incidents, feedback, eval runs
 - Artifacts: vector index + model binaries (versioned via DVC/MLflow)
- Monitoring: Prometheus metrics + Grafana dashboards

- Batch: DVC pipeline + Prefect scheduled flows (reindex, drift, eval, ret)

Data Flow

Online (request):

- 1) User submits alert + logs
- 2) Online featurizer templates logs -> fixed vocab vector
- 3) IsolationForest scores anomaly
- 4) Hybrid retrieval returns top-K runbook chunks
- 5) Agent drafts response in JSON, requiring evidence *doc_ids per action*
- 6) *UI renders actions + evidence; user submits feedback*

Offline (pipelines):

- Download -> parse logs -> build features -> train anomaly model -> build

Key Design Decisions

- CPU-only: small local LLM; rely on retrieval grounding and templates
- Evidence enforcement: hard-coded validator rejects actions without *doc_id*
- Hybrid retrieval: BM25 improves exact-match alert retrieval; vectors improve
- Versioning: DVC tracks datasets/artifacts; MLflow tracks runs/metrics/req

Reliability & Safety

- Fallback mode: *LLM_PROVIDER=mock* for deterministic outputs (tests, limits)
- Timeouts/circuit breakers around LLM calls
- Prompt injection mitigation: treat retrieved text as untrusted; strip un
- No destructive commands or automated execution

Observability

Prometheus metrics:

- request rate, latency histograms, 5xx counts
- retrieval latency, anomaly scoring latency

Dashboards:

- API overview, endpoint breakdown, p95 latency

Drift & Retraining

- Drift reports via Evidently on feature distribution
- Threshold-based alert: recommend retrain
- Retrain pipeline is triggered manually or via Prefect schedule (optional)

Testing Strategy

- Unit: featurizer, anomaly scoring, retriever
- Safety: validator blocks ungrounded actions
- Integration: /incident/analyze end-to-end with mock LLM
- CI: ruff + pytest + (optional) mypy

Future Work

- Incident clustering/correlation
- Dedicated vector DB (Qdrant) and reranking
- Role-based access and audit logs
- Integrations: Slack/Jira export

 Copy code

50. “Polish pack” (small files that add huge credibility)

50.1 docs/architecture.md

- paste the ASCII diagram
- add offline vs online sequence diagrams (text)

50.2 docs/demo_script.md

- step-by-step: what to click, what to show, what to say (2 min)

50.3 README.md badges

- CI badge
- Docker badge (optional)

51. Final: the exact “portfolio story” you should tell (30 seconds)

“OpsPilot is an incident-response copilot that combines classical AIOps anomaly detection with RAG over real runbooks and a tool-using agent. The key is reliability: every action is evidence-backed, and pipelines are reproducible via DVC/MLflow. It runs CPU-only with local LLM support, ships as Docker Compose, and includes Prometheus/Grafana observability plus drift monitoring.”

If you want to push this into the **top 1%** tier, tell me your target domain preference:

- **SRE/AIOps** (current project)
- **FinTech** (fraud/AML copilot)
- **Healthcare** (clinical guideline assistant)
- **Legal/Compliance** (policy QA + redlining)
- **E-commerce** (support + returns automation)

I can adapt OpsPilot into the same blueprint with domain-specific datasets and “company-like” features while keeping the same MLOps backbone.



tell me the steps like first this then this and then do it very carefully and in detail

Below is the **exact “do this → then this → then this” build order** to finish OpsPilot end-to-end with **no ambiguity**. Follow it sequentially.

Step 0 — Prereqs (do once)

1. Install:
 - **Python 3.11**
 - **Git**
 - **Docker + Docker Compose**
2. Optional (recommended for local LLM):
 - **Ollama** (desktop install)

Then later you'll run: `ollama pull llama3.2:3b-instruct-q4_K_M`

Step 1 — Create repo + baseline structure

```
bash
```

```
mkdir opspilot && cd opspilot
git init
mkdir -p src/ospilot/{api/routes,agent,rag,anomaly,storage,observability,}
```



```
scripts/{data,features,train,rag,eval} \
ui tests docker data/{raw,processed,features,eval} artifacts mode
```

 Copy code

Create these files (empty for now):

- `pyproject.toml`
- `.env.example`
- `docker-compose.yml`
- `docker/api.Dockerfile`
- `docker/ui.Dockerfile`
- `docker/prometheus.yml`
- `drain3.ini`
- `Makefile`
- `README.md`

Commit:

```
bash

git add .
git commit -m "Scaffold OpsPilot repo"
```

 Copy code

Step 2 — Add dependencies + install locally

1. Paste the `pyproject.toml` content (from earlier).
2. Create venv + install:

```
bash
```

```
python -m venv .venv
source .venv/bin/activate
pip install -U pip
pip install -e ".[api,ui]"
```

 Copy code

Quick check:

```
bash
```

```
python -c "import fastapi, sklearn, sentence_transformers; print('done ok!')"
```

 Copy code

Commit:

```
bash
```

```
git add pyproject.toml
git commit -m "Add dependencies"
```

 Copy code

Step 3 — Add env + docker files (so reviewers can run it)

1. Paste `.env.example`, copy to `.env`:

```
bash
```

```
cp .env.example .env
```

 Copy code

2. Paste `docker-compose.yml`, `docker/api.Dockerfile`, `docker/ui.Dockerfile`, `docker/prometheus.yml`.

Commit:

```
bash
```

```
git add .env.example docker-compose.yml docker/ docker/*.Dockerfile  
git commit -m "Add Docker Compose stack"
```

 Copy code

Step 4 — Implement the API skeleton (health first)

Create and paste:

- `src/opspilot/api/main.py`
- `src/opspilot/api/routes/health.py`
- `src/opspilot/observability/logging.py`
- `src/opspilot/observability/metrics.py`

Run:

```
bash
```

```
uvicorn opspilot.api.main:app --reload --port 8000
```

 Copy code

Test:

- Open: `http://localhost:8000/health`
- Open: `http://localhost:8000/docs`

Commit:

```
bash
```

```
git add src/opspilot/api src/opspilot/observability
git commit -m "Add FastAPI skeleton + metrics/logging"
```

 Copy code

Step 5 — Download public data (runbooks + loghub subset)

Add:

- `scripts/data/download_all.py`

Run:

```
bash
```

```
python scripts/data/download_all.py
```

 Copy code

Verify:

```
bash
```

```
ls data/raw/runbooks | head
```

```
ls data/raw/loghub_subset/hdfs/HDFS.log | cat | head -n 3
```

 Copy code

Now DVC (optional now, but do it early if you can):

```
bash
```

```
dvc init
```

```
dvc add data/raw
```

```
git add data/.gitignore data/raw.dvc .dvc
```

```
git commit -m "Track raw data with DVC"
```

 Copy code

Step 6 — Build the RAG index (FAISS) from runbooks (offline)

Add and paste:

- `src/opspilot/embeddings/encoder.py`
- `src/opspilot/rag/chunking.py`
- `src/opspilot/rag/index.py`
- `scripts/rag/build_index.py`

Run:

```
bash
```

```
python scripts/rag/build_index.py
```

 Copy code

Verify artifacts exist:

```
bash
```

```
ls artifacts/vector_index  
# expect: index.faiss + meta.jsonl
```

 Copy code

Commit:

```
bash
```

```
git add src/opspilot/embeddings src/opspilot/rag scripts/rag artifacts/vec  
git commit -m "Build FAISS runbook index"
```

 Copy code

Step 7 — Add DocStore + Hybrid Retriever (vector + BM25)

Add and paste:

- `src/opspilot/rag/docstore.py`
- `src/opspilot/rag/bm25.py`
- `src/opspilot/rag/retriever.py` (final hybrid version)

Then add RAG API endpoint:

- `src/opspilot/api/schemas.py` (include `RagSearchRequest` + `RetrievedChunk`)
- `src/opspilot/api/routes/rag.py`

Update `main.py` to include rag router.

Run API and test:

```
bash
```

```
uvicorn opspilot.api.main:app --reload --port 8000
curl -X POST http://localhost:8000/rag/search \
  -H "Content-Type: application/json" \
  -d '{"query":"NodeFilesystemSpaceFillingUp disk", "top_k": 3, "filter_type":
```

 Copy code

Commit:

```
bash
```

```
git add src/opspilot/rag src/opspilot/api
git commit -m "Add docstore + hybrid RAG retriever + /rag/search"
```

 Copy code

Step 8 — Build anomaly pipeline OFFLINE (Drain3 templates → features → train IsolationForest)

Add and paste:

- `drain3.ini`
- `scripts/features/parse_logs.py`
- `scripts/features/build_features.py` (must also write `artifacts/anomaly_vocab.json`)
- `scripts/train/train_anomaly.py`

Run:

```
bash
```

```
python scripts/features/parse_logs.py
python scripts/features/build_features.py
python scripts/train/train_anomaly.py
```

 Copy code

Verify:

```
bash
```

```
ls data/processed/log_templates.parquet
ls data/features/log_windows.parquet
ls artifacts/anomaly_vocab.json
ls models/anomaly_model.pkl
```

 Copy code

Commit:

```
bash
```

```
git add scripts/features scripts/train drain3.ini artifacts/anomaly_vocab.  
git commit -m "Add offline anomaly pipeline (Drain3 + Isolation)"
```

[Copy code](#)

Step 9 — Implement ONLINE anomaly scoring endpoint

Add and paste:

- `src/opspilot/anomaly/features.py` (OnlineLogFeaturizer)
- `src/opspilot/anomaly/infer.py` (AnomalyScorer)
- `src/opspilot/api/routes/anomaly.py`

Update `main.py` to include anomaly router.

Run and test:

```
bash
```

```
uvicorn opspilot.api.main:app --reload --port 8000
```

```
curl -X POST http://localhost:8000/anomaly/score \  
-H "Content-Type: application/json" \  
-d '{"log_lines":["ERROR disk full","WARN retrying"],"window_minutes":5}'
```

[Copy code](#)

Commit:

```
bash
```



```
git add src/opspilot/anomaly src/opspilot/api/routes/anomaly.py
git commit -m "Add online anomaly scoring endpoint"
```

Step 10 — Add SQL storage for feedback

Add and paste:

- `src/opspilot/storage/db.py`
- `src/opspilot/storage/models.py`
- `src/opspilot/api/routes/feedback.py`
- add FeedbackRequest schema in `src/opspilot/api/schemas.py`

Run:

bash

```
uvicorn opspilot.api.main:app --reload --port 8000
curl -X POST http://localhost:8000/feedback \
  -H "Content-Type: application/json" \
  -d '{"incident_id":"INC-1","helpful":true,"tags":["good"],"comment":"nice"'
```

Commit:

bash

```
git add src/opspilot/storage src/opspilot/api/routes/feedback.py
git commit -m "Add feedback storage"
```

Step 11 — Implement the agent (LangGraph) and enforce groundedness

Add and paste:

- `src/opspilot/agent/prompts.py`
- `src/opspilot/agent/safety.py`
- `src/opspilot/agent/tools.py`
- `src/opspilot/agent/graph.py`

Important: Start with `LLM_PROVIDER=mock` so everything works deterministically.

Now implement `/incident/analyze`:

- `src/opspilot/api/routes/incident.py`

Update `main.py` to include incident router.

Test end-to-end:

```
bash
```

```
export LLM_PROVIDER=mock
```

```
uvicorn opspilot.api.main:app --reload --port 8000
```

```
curl -X POST http://localhost:8000/incident/analyze \
```

```
-H "Content-Type: application/json" \
```

```
-d '{"incident_id":"INC-2","alert_title":"NodeFilesystemSpaceFillingUp",'
```

 Copy code

Commit:

```
bash
```

```
git add src/opspilot/agent src/opspilot/api/routes/incident.py
git commit -m "Add LangGraph agent + incident analysis endpoint"
```

[Copy code](#)

Step 12 — Enable local LLM (optional, but the final wow factor)

1. Install Ollama and pull a small model:

bash

```
ollama pull llama3.2:3b-instruct-q4_K_M
```

[Copy code](#)

2. Set env in `.env`:

bash

```
LLM_PROVIDER=ollama
OLLAMA_HOST=http://localhost:11434
OLLAMA_MODEL=llama3.2:3b-instruct-q4_K_M
```

[Copy code](#)

3. Run:

bash

```
uvicorn opspilot.api.main:app --reload --port 8000
```

[Copy code](#)

Test `/incident/analyze` again and confirm it returns JSON.

If JSON parsing fails occasionally:

- tighten prompt (already strict)

- add "JSON-only extraction" (already included)
- keep fallback: if parse fails, return safe template response

Commit:

```
bash

git add .env.example
git commit -m "Enable Ollama local LLM mode"
```

 Copy code

Step 13 — Build Streamlit UI and connect to API

Add `ui/streamlit_app.py` and run:

```
bash

streamlit run ui/streamlit_app.py
```

 Copy code

Click:

- Submit incident
- Verify anomaly + retrieved evidence + postmortem output
- Submit feedback

Commit:

```
bash

git add ui/streamlit_app.py
git commit -m "Add Streamlit incident console"
```

 Copy code

Step 14 — Add evaluation (RAG metrics) + seed gold set

Add:

- `data/eval/rag_gold.jsonl` (seed with 20 lines)
- `scripts/eval/run_eval.py`

Run:

```
bash
```

```
python scripts/eval/run_eval.py  
cat reports/rag_metrics.json
```

 Copy code

Optional MLflow:

```
bash
```

```
mlflow ui --backend-store-uri sqlite:///mlruns.db --default-artifact-root
```

 Copy code

Commit:

```
bash
```

```
git add data/eval scripts/eval reports/rag_metrics.json  
git commit -m "Add RAG evaluation harness"
```

 Copy code

Step 15 — Add tests (minimum suite) + CI

Add tests:

- `tests/test_api_contract.py`
- `tests/test_agent_safety.py`

Run:

```
bash
```

```
pytest -q
```

 Copy code**Add CI:**

- `.github/workflows/ci.yml`

Commit:

```
bash
```

```
git add tests .github/workflows/ci.yml  
git commit -m "Add tests + CI workflow"
```

 Copy code

Step 16 — Observability stack: Prometheus + Grafana dashboard

1. Ensure `instrument_app` exposes `/metrics` (already done).
2. Run full docker stack:

```
bash
```

```
docker compose up --build
```

 Copy code

3. Open:

- Prometheus: `http://localhost:9090`
- Grafana: `http://localhost:3000` (default admin/admin in many images)

Import dashboard JSON:

- `docker/grafana_dashboard_opspilot.json` (create file from earlier)

Commit:

```
bash

git add docker/grafana_dashboard_opspilot.json
git commit -m "Add Grafana dashboard"
```

 Copy code

Step 17 — Prefect flows (nightly eval, drift report, reindex)

Add:

- `src/ospilot/workflows/prefect_flows.py`
- `src/ospilot/workflows/drift.py`

Run:

```
bash

prefect server start
prefect worker start --pool default-agent-pool
python -c "from ospilot.workflows.prefect_flows import nightly; nightly()"
python src/ospilot/workflows/drift.py
```

 Copy code

Verify:

- `reports/drift_report.html`

Commit:

```
bash
```

```
git add src/opsilot/workflows reports/drift_report.html  
git commit -m "Add Prefect workflows + drift report"
```

 Copy code

Step 18 — Final packaging + README polish (what makes it “flagship”)

Create docs:

- `docs/system_design.md`
- `docs/demo_script.md`

Update `README.md`:

- quickstart
- architecture diagram
- evaluation results (your real metrics)
- screenshots / GIF checklist

Tag release:

```
bash
```

```
git tag v0.1.0  
git push --tags
```

 Copy code

The “one command” run paths (what recruiters love)

Full local stack

```
bash
```

```
docker compose up --build
```

 Copy code

Rebuild everything reproducibly

```
bash
```

```
dvc repro
```

 Copy code

Run eval

```
bash
```

```
python scripts/eval/run_eval.py
```

 Copy code

If you want, I can also give you a single master checklist file (docs/build_steps.md) you can paste into the repo so anyone can follow it exactly like a runbook (that itself becomes part of the project’s “production vibe”).

   ...

[yes continue](#)

52. Paste-ready master build checklist (docs/build_steps.md)

Create docs/build_steps.md and paste this exactly. This becomes your project runbook and is a *huge* credibility signal.

md

OpsPilot Build Steps (Runbook)

This document is the exact step-by-step guide to build, run, and evaluate (

0) Prerequisites

Install:

- Python 3.11
- Git
- Docker + Docker Compose

Optional (for local LLM):

- Ollama (<https://ollama.com>)

1) Clone + environment setup

```
```bash
git clone <YOUR_REPO_URL> opspilot
cd opspilot

python -m venv .venv
source .venv/bin/activate
pip install -U pip
pip install -e "[api,ui]"
```

 Copy code

## 2) Configure environment variables

```
bash
```

```
cp .env.example .env
```

 Copy code

Optional local LLM mode:

```
bash
```

```
install ollama first
ollama pull llama3.2:3b-instruct-q4_K_M

in .env:
LLM_PROVIDER=ollama
OLLAMA_HOST=http://localhost:11434
OLLAMA_MODEL=llama3.2:3b-instruct-q4_K_M
```

 Copy code

For deterministic dev/testing:

```
bash
```

```
in .env:
```

```
LLM_PROVIDER=mock
```

[Copy code](#)

---

### 3) Download public datasets

Downloads:

- Loghub subset (HDFS logs)
- Prometheus Operator runbooks

```
bash
```

```
python scripts/data/download_all.py
```

[Copy code](#)

Verify:

```
bash
```

```
ls data/raw/runbooks | head
```

```
ls data/raw/loghub_subset/hdfs/HDFS.log | head -n 3
```

[Copy code](#)

---

### 4) Build RAG index (offline)

Build FAISS index from runbooks:

```
bash
```

```
python scripts/rag/build_index.py
```

[Copy code](#)

Verify:

```
bash
```

```
ls artifacts/vector_index
```

```
Expect: index.faiss, meta.jsonl
```

[Copy code](#)

---

## 5) Build anomaly pipeline (offline)

A) Parse logs into templates:

```
bash
```

```
python scripts/features/parse_logs.py
```

[Copy code](#)

B) Build window features + vocab:

```
bash
```

```
python scripts/features/build_features.py
```

[Copy code](#)

C) Train anomaly model:

```
bash
```

```
python scripts/train/train_anomaly.py
```

[Copy code](#)

Verify:

bash

```
ls data/processed/log_templates.parquet
ls data/features/log_windows.parquet
ls artifacts/anomaly_vocab.json
ls models/anomaly_model.pkl
```

 Copy code

## 6) Run API locally

bash

```
uvicorn opspilot.api.main:app --reload --port 8000
```

 Copy code

Open:

- <http://localhost:8000/health> ↗
- <http://localhost:8000/docs> ↗

Test endpoints:

RAG:

bash

```
curl -X POST http://localhost:8000/rag/search \
-H "Content-Type: application/json" \
-d '{"query": "disk pressure NodeFilesystemSpaceFillingUp", "tokens": ["disk", "pressure", "NodeFilesystemSpaceFillingUp"]}'
```

 Copy code

### Anomaly:

bash

```
curl -X POST http://localhost:8000/anomaly/score \
-H "Content-Type: application/json" \
-d '{"log_lines":["ERROR disk full","WARN retrying"],"window_minutes":5}'
```

 Copy code

### Incident analyze:

bash

```
curl -X POST http://localhost:8000/incident/analyze \
-H "Content-Type: application/json" \
-d '{"incident_id":"INC-1","alert_title":"NodeFilesystemSpaceFillingUp",'
```

 Copy code

## 7) Run Streamlit UI

In a new terminal:

bash

```
source .venv/bin/activate
streamlit run ui/streamlit_app.py
```

 Copy code

Open:

- <http://localhost:8501> ↗

## 8) Run evaluation

Seed evaluation dataset:

- `data/eval/rag_gold.jsonl`

Run:

```
bash
```

```
python scripts/eval/run_eval.py
cat reports/rag_metrics.json
```

 Copy code

---

## 9) Run full stack via Docker Compose

```
bash
```

```
docker compose up --build
```

 Copy code

Open:

- UI: <http://localhost:8501> ↗
- API: <http://localhost:8000/docs> ↗
- Prometheus: <http://localhost:9090> ↗
- Grafana: <http://localhost:3000> ↗

---

## 10) Drift monitoring (optional)

```
bash
```



```
python src/opspilot/workflows/drift.py
ls reports/drift_report.html
```

 Copy code

---

## 11) Reproducible rebuild (optional DVC)

```
bash
```

```
dvc repro
```

 Copy code

---

## 12) Common troubleshooting

### A) Ollama JSON parsing errors

- Ensure prompt enforces JSON output.
- Use LLM\_PROVIDER=mock to validate system end-to-end.
- Use smaller outputs and top\_k<=6.

### B) FAISS dimension mismatch

- Ensure embedding model is all-MiniLM-L6-v2 (384 dims).
- Rebuild index after switching models:

```
bash
```

```
rm -rf artifacts/vector_index
python scripts/rag/build_index.py
```

 Copy code

## C) Missing runbooks

- Confirm `data/raw/runbooks` exists and has `.md` files.
- Re-run:

```
bash
```

```
python scripts/data/download_all.py
```

[Copy code](#)

## D) Drain3 not installed

- Ensure deps installed:

```
bash
```

```
pip install drain3
```

[Copy code](#)

```
bash
```

```

```

```
53. Add "single-command onboarding" scripts (very recruiter-friendly)
```

```
53.1 `scripts/bootstrap.sh`
```

```
```bash
```

```
#!/usr/bin/env bash
```

```
set -euo pipefail
```

```
python -m venv .venv
```

```
source .venv/bin/activate
```

```
pip install -U pip
pip install -e ".[api,ui]"

cp -n .env.example .env || true
python scripts/data/download_all.py
python scripts/rag/build_index.py
python scripts/features/parse_logs.py
python scripts/features/build_features.py
python scripts/train/train_anomaly.py

echo "Bootstrap complete."
echo "Run API: uvicorn opspilot.api.main:app --reload --port 8000"
echo "Run UI: streamlit run ui/streamlit_app.py"
```

 Copy code

Make executable:

```
bash

chmod +x scripts/bootstrap.sh
```

 Copy code

53.2 scripts/run_local.sh

```
bash

#!/usr/bin/env bash
set -euo pipefail
source .venv/bin/activate
uvicorn opspilot.api.main:app --reload --port 8000
```

 Copy code

53.3 scripts/run_ui.sh

bash

```
#!/usr/bin/env bash
set -euo pipefail
source .venv/bin/activate
streamlit run ui/streamlit_app.py
```

 Copy code

Commit:

bash

```
git add docs/build_steps.md scripts/bootstrap.sh scripts/run_local.sh scri
git commit -m "Add build runbook + bootstrap scripts"
```

 Copy code

54. Add “examples” payloads (so reviewers can demo instantly)

Create:

54.1 examples/incident_node_disk.json

json

```
{
  "incident_id": "INC-2026-00012",
  "alert_title": "NodeFilesystemSpaceFillingUp",
  "service": "kubernetes",
  "environment": "prod",
  "log_lines": [
    "ERROR: disk full on /var/lib/containerd",
```

```
"WARN: eviction manager: pods evicted due to disk pressure",  
"ERROR: write failed: no space left on device"  
],  
"user_notes": "Pods failing scheduling; nodes show disk pressure."  
}
```

 Copy code

54.2 examples/incident_crashloop.json

json

```
{  
  "incident_id": "INC-2026-00013",  
  "alert_title": "KubePodCrashLooping",  
  "service": "kubernetes",  
  "environment": "prod",  
  "log_lines": [  
    "ERROR: container exited with code 137",  
    "Exception: database connection timeout",  
    "WARN: restarting container backoff"  
  ],  
  "user_notes": "Checkout service pods restart repeatedly."  
}
```

 Copy code

Demo curl commands (add to README too):

bash

```
curl -X POST http://localhost:8000/incident/analyze \  
-H "Content-Type: application/json" \  
-d @examples/incident_node_disk.json
```

 Copy code

Commit:

```
bash

git add examples/
git commit -m "Add demo example incidents"
```

 Copy code

55. Add admin endpoints safely (without security footguns)

Even if you don't enable auth by default, implement admin endpoints **behind a role check** (works as admin in dev, and enforce JWT later).

1. Add:

- `src/opspilot/api/deps.py` (from earlier)
- `src/opspilot/api/routes/admin.py`

2. Include router in `main.py` with prefix `/admin`

Test (dev mode has admin user if `ENABLE_AUTH=false`):

```
bash

curl -X POST http://localhost:8000/admin/reindex
curl -X POST http://localhost:8000/admin/retrain
```

 Copy code

Commit:

```
bash

git add src/opspilot/api/deps.py src/opspilot/api/routes/admin.py
git commit -m "Add admin endpoints with RBAC dependency"
```

 Copy code

56. Final “ship checklist” (before you publish on GitHub)

Must-have (don't skip)

- ☐ `docker compose up --build` works on a fresh machine
- ☐ `scripts/bootstrap.sh` completes without manual edits
- ☐ `/incident/analyze` works with `LLM_PROVIDER=mock`
- ☐ RAG index builds and returns chunks
- ☐ `scripts/eval/run_eval.py` produces `reports/rag_metrics.json`
- ☐ `pytest -q` passes
- ☐ `.github/workflows/ci.yml` passing badge on README
- ☐ screenshots: UI, /docs, MLflow (if used), Grafana dashboard

Nice-to-have

- ☐ add a GIF walkthrough in README
- ☐ add a short “Design decisions” section

57. How to present this as “one flagship project replacing 3–4 projects”

When someone asks “What did you build?” you answer in this order:

1. **Product:** “Incident response copilot to reduce MTTR”
2. **AI:** “RAG over runbooks + agent orchestration + anomaly detection”
3. **Reliability:** “evidence enforcement + evaluations + tests”
4. **MLOps:** “DVC pipelines + MLflow + Docker”
5. **Ops:** “Prometheus/Grafana + drift monitoring”

That narrative is extremely convincing.

If you want one more continue: I can provide a **paste-ready README.md** that's structured like a real production repo (with badges, demo GIF placeholders, architecture diagram section, evaluation table, and deploy instructions).



continue and also answer is all the things and every dataset and all other things available?

Thought for 6s >

Paste-ready `README.md` (flagship-quality)

Create/replace `README.md` with this:

md

```
# OpsPilot — Incident Response Copilot (AIOps + RAG + Tool-Using Agents)
```

OpsPilot is a production-style incident response copilot that reduces MTTR

- **Log anomaly detection** (Drain3 template mining + IsolationForest)
- **Hybrid RAG** over real runbooks (BM25 + FAISS vector search)
- A **tool-using agent** (LangGraph) that produces **evidence-backed** mit.


```
- **MLOps + Observability**: DVC pipelines, optional MLflow tracking, Promi  
  
> Runs on CPU, uses open-source tooling, and avoids paid APIs.  
  
---  
  
## Demo (2 minutes)  
  
### Local quickstart  
```bash  
cp .env.example .env
./scripts/bootstrap.sh
uvicorn opspilot.api.main:app --reload --port 8000
streamlit run ui/streamlit_app.py
```

 Copy code

Open:

- UI: <http://localhost:8501> ↗
- API docs: <http://localhost:8000/docs> ↗

## One-command full stack

```
bash
```

```
docker compose up --build
```

 Copy code

Open:

- UI: <http://localhost:8501> ↗
- API docs: <http://localhost:8000/docs> ↗
- Prometheus: <http://localhost:9090> ↗

- Grafana: <http://localhost:3000> ↗
- 

## Architecture

### Online flow (request-time)

1. User submits incident (alert + logs)
2. **Online featurizer** (Drain3) converts logs into template-frequency vector
3. **Anomaly model** scores severity signal
4. **Hybrid retriever** returns top-K runbook chunks (BM25 + FAISS)
5. **Agent (LangGraph)** orchestrates tools and returns structured JSON:
  - summary, hypotheses, mitigation steps, verification, fallback plan, postmortem draft
6. **Safety guardrail**: every action must cite evidence `doc_id` from retrieved context

### Offline flow (batch / reproducible)

- Download data → parse logs → build features → train anomaly model → build RAG index → evaluate → publish artifacts
  - Drift monitoring and scheduled runs via Prefect (optional)
- 

## Features

-  CPU-friendly: small local model support (Ollama) or deterministic mock mode
-  Evidence enforcement (hard validator rejects ungrounded actions)
-  Reproducible pipeline (DVC stages)
-  Monitoring (Prometheus metrics + Grafana dashboard)

-  Drift report (Evidently HTML)
-  Tests + CI

---

## Endpoints

- GET /health
- POST /rag/search
- POST /anomaly/score
- POST /incident/analyze
- POST /feedback
- POST /admin/reindex (optional)
- POST /admin/retrain (optional)

---

## Example usage

### Analyze incident

```
bash
```

```
curl -X POST http://localhost:8000/incident/analyze \
-H "Content-Type: application/json" \
-d @examples/incident_node_disk.json
```

 Copy code

### Search runbooks

```
bash
```

```
curl -X POST http://localhost:8000/rag/search \
 -H "Content-Type: application/json" \
 -d '{"query": "NodeFilesystemSpaceFillingUp disk pressure", "{'
```

 Copy code

---

## Evaluation

Run retrieval evaluation:

```
bash

python scripts/eval/run_eval.py
cat reports/rag_metrics.json
```

 Copy code

Metrics:

- Recall@K, MRR using `data/eval/rag_gold.jsonl`

---

## Observability

- Prometheus metrics: `GET /metrics`
- Grafana dashboard JSON: `docker/grafana_dashboard_opspilot.json`

---

## Reproducible build (DVC)

```
bash
```

```
dvc repro
```

## Data sources & licensing (important)

This project uses public/open resources:

- **Loghub (LogPAI)** system logs (research/academic usage; requires citation)
- **Prometheus Operator Runbooks** (Apache-2.0)
- **Drain3** (open-source; used for template mining)

See `docs/data_licenses.md` for details.

ChatGPT 5.2 Thinking ▾

 Temporary Chat ...

- `docs/build_steps.md` — exact step-by-step build runbook
- `docs/system_design.md` — one-page RFC style design
- `docs/demo_script.md` — what to show in interviews

## License

Project code: MIT (or your choice).

Datasets: see original dataset licenses; do not redistribute if restricted.

```
yaml
```

```

```

```
Create `docs/data_licenses.md` (answering your dataset question clearly.
```

You asked: **\*\*\*"is everything / every dataset available?"\*\***

Yes, **\*\*the core datasets and tools are publicly accessible\*\***, but **\*\*some h**

Create `docs/data\_licenses.md`:

```
```md
```

```
# Data & Tool Licenses / Availability
```

This project uses public/open resources. Always follow original terms.

```
## 1) Loghub (LogPAI) – system logs
```

Availability:

- Public repo with many log datasets.

Important terms:

- Loghub datasets are “freely available for research or academic work” and

Source: Loghub license text. :contentReference[oaicite:0]{index=0}

Practical guidance:

- Do NOT commit large raw logs to GitHub.
- Use the provided `scripts/data/download\_all.py` to fetch data.
- If you publish results, add citations in README.

```
## 2) Prometheus Operator Runbooks
```

Availability:

- Public GitHub repo; official runbooks for alerts.

License:

- Repo indicates Apache-2.0. :contentReference[oaicite:1]{index=1}

Practical guidance:

- You can safely index and use the markdown content for RAG.
- Prefer downloading at build time rather than copying whole repo.

*## 3) Drain3***Availability:**

- Public GitHub repo; open-source streaming log template miner. :contentReference[oaicite:2]{index=1}

Practical guidance:

- Used for parsing logs into templates both offline and online.

*## 4) Optional dataset: PAG_DB (Synthetic Cloud Incident Dataset)***Availability:**

- Public GitHub repo; synthetic/anonymized incidents for CAPM (actions) generated by Loghub

Practical guidance:

- Optional; only needed if you want richer postmortem/action evaluation beyond CAPM
- Check repo license/terms before redistribution (prefer downloading)  Copy code 

Straight answer to your question

- **Yes**, the project is buildable with **publicly available** datasets and open-source tools.
- **However**, Loghub's license emphasizes **research/academic use + citation**.  +1
- The runbooks are **Apache-2.0**, so they're broadly usable.  +1
- Optional PAG dataset is public (synthetic incidents), but you should verify its license before redistributing it. 

Best practice for your GitHub repo: don't upload raw datasets; provide **download scripts + DVC** pointers.

Final “do this next” checklist (so you don't miss anything)

1. Add `README.md` (above)
2. Add `docs/data_licenses.md` (above)
3. Run:

```
bash
```

```
./scripts/bootstrap.sh  
pytest -q  
python scripts/eval/run_eval.py  
docker compose up --build
```

 Copy code

4. Take screenshots:
 - UI result
 - `/docs`
 - Grafana dashboard
 - `reports/rag_metrics.json` output

If you want, paste your current folder tree (or GitHub link) and I'll point out exactly what's missing to make it “ship-ready.”

     Sources

so everything is completed from your end like if i use all the above chhats than will the project be fully complete?

If you implement **everything we discussed**, you will have a **fully working, end-to-end project**: data download → offline pipelines (RAG index + anomaly model) → online API (FastAPI) → agent orchestration → UI (Streamlit) → evaluation → monitoring → drift report → Docker deployment → CI tests.

But I want to be 100% honest: **this chat is a complete blueprint**, not a guarantee that every pasted snippet will run *without any small wiring fixes*, because:

- dataset folder structures can differ slightly (especially **Loghub** paths), so `download_all.py` may need a minor path tweak the first time,
- some Python packages can have version quirks across OS (especially FAISS on Windows),
- Grafana datasource UID mapping is usually a one-time manual step,
- Ollama JSON output can vary by model, so you may occasionally need to tighten JSON extraction or keep `LLM_PROVIDER=mock` for tests.

What “fully complete” means here

✅ You will have a **portfolio-grade system** that:

- runs locally (and via Docker Compose),
- uses only public data + open-source tools,

- demonstrates RAG + agent tools + classical ML + MLOps + monitoring,
- is interview-demo-able with scripts + examples + docs.

What you still must do (small but required)

To make it *truly* “no surprises”, you should do these final verification steps:

1. Run the bootstrap

```
bash
```

```
./scripts/bootstrap.sh
```

[Copy code](#)

2. Run the API + UI

```
bash
```

```
uvicorn opspilot.api.main:app --reload --port 8000  
streamlit run ui/streamlit_app.py
```

[Copy code](#)

3. Run tests

```
bash
```

```
pytest -q
```

[Copy code](#)

4. Run eval

```
bash
```

```
python scripts/eval/run_eval.py
```

[Copy code](#)

5. Run Docker stack

```
bash
```

```
docker compose up --build
```

 Copy code

If all 5 succeed, then yes—the **project is complete in the real sense**.

The “completion checklist” (what a reviewer expects)

You’re complete when you have:

- `README.md` + `docs/build_steps.md` + `docs/system_design.md`
- `scripts/bootstrap.sh` works on a clean machine
- `/incident/analyze` works in **mock mode** (always) and ideally also in **ollama mode**
- `reports/rag_metrics.json` exists with non-zero queries
- Prometheus `/metrics` exposes values and Grafana dashboard can be imported
- CI workflow passes on GitHub

Quick verdict

- **Blueprint completeness:**  yes, it’s end-to-end and production-style.
- **Copy/paste “will run 100% without any adjustment”:** usually yes, but expect **1–3 tiny environment/path fixes** depending on OS and the Loghub file layout.

If you want the most bulletproof finish: paste your OS (Windows/macOS/Linux) and whether you’ll use Ollama, and I’ll give you a final “platform-specific” patch list (no questions beyond that) to make it run first try.



16. Concrete `docker-compose.yml` (full stack, local-first)

Create `docker-compose.yml` at repo root:

yml

 Copy code

```
version: "3.9"
```

```
services:
```

```
  postgres:
```

```
    image: postgres:16-alpine
```

